

메가존클라우드 Solution Architecture

기술가이드

고가용성 서버 전환 구축 사업

곽덕연

2025-3-18

[목차]

고가용성 서버 전환 구축 사업

Background.....	3
개념 설명	6
사용 기술 선정 이유	9
가. WEB 서버: NGINX(LB), Redis.....	9
나. WAS 서버: Apache + php.....	9
다. DB 서버: MariaDB	10
라. ELK state	10
마. Metricbeats.....	10
<전제: 외부연결및 패키지 설치>	11
흐름00. 각 VM 생성	11
흐름01. 서버용 VM에 패키지 설치하기.....	13
흐름02. 각 VM 사설망 전환.....	16
흐름03. 라우터 설정하기	16
3티어 구축하기	18
< DB 서버 구축 >	18
흐름04. IP 및 라우팅 설정.....	18
흐름05. MariaDB 설정 변경	18

< WEB 서버 및 Redis 설정 >	21
흐름06. IP 및 라우팅 설정	21
흐름07. WEB엔진 Nginx 설정	22
흐름08. Redis DB 설정	23
 < WAS 설정 및 이중화/세션유지 >	 25
흐름09. IP 및 라우팅 설정	25
흐름10. Apache 기본 사이트 설정	26
흐름11. Redis 세션 설정 (WAS1, WAS2 모두)	26
흐름12. 테스트	27
 < 관리용 BastionHost/ELK 서버 설정 >	 30
흐름13. IP 및 라우팅 설정	30
흐름14. Elasticsearch 설정	31
흐름15. Logstash 설정 (로그 수집)	32
흐름16. Kibana 설정	34
흐름17. 클라이언트 서버에서 Rsyslog 설정	35
 < ELK 사용해보기 >	 36
흐름18. Kibana 접속하기	36
흐름19. Rsyslog로 수집된 정보 확인	37
흐름20. 로깅 정책 적용하기	40
흐름21. 서버 헬스체크 플러그인(Metricbeat) 설치	43

Background

■ 본 실습은 3티어 아키텍처 구성 및 ELK로 모니터링을 목적으로 한다.

- 3티어 구성: WEB서버(nginx) , WAS(Apache2+php), DB서버(MariaDB)
- ELK는 Elasticsearch, logstash, Kibana를 말한다.

■ 본 기술 가이드는 실제(물리적)환경에서 진행하는 것이 아닌 VM와 GNS3를 통한 가상 시나리오의 시뮬레이션이다.

- 서버는 오라클사의 Virtual Box를 이용한 가상머신(VM)을 이용한다.
- 라우터는 GNS3 프로그램으로 대체한다.

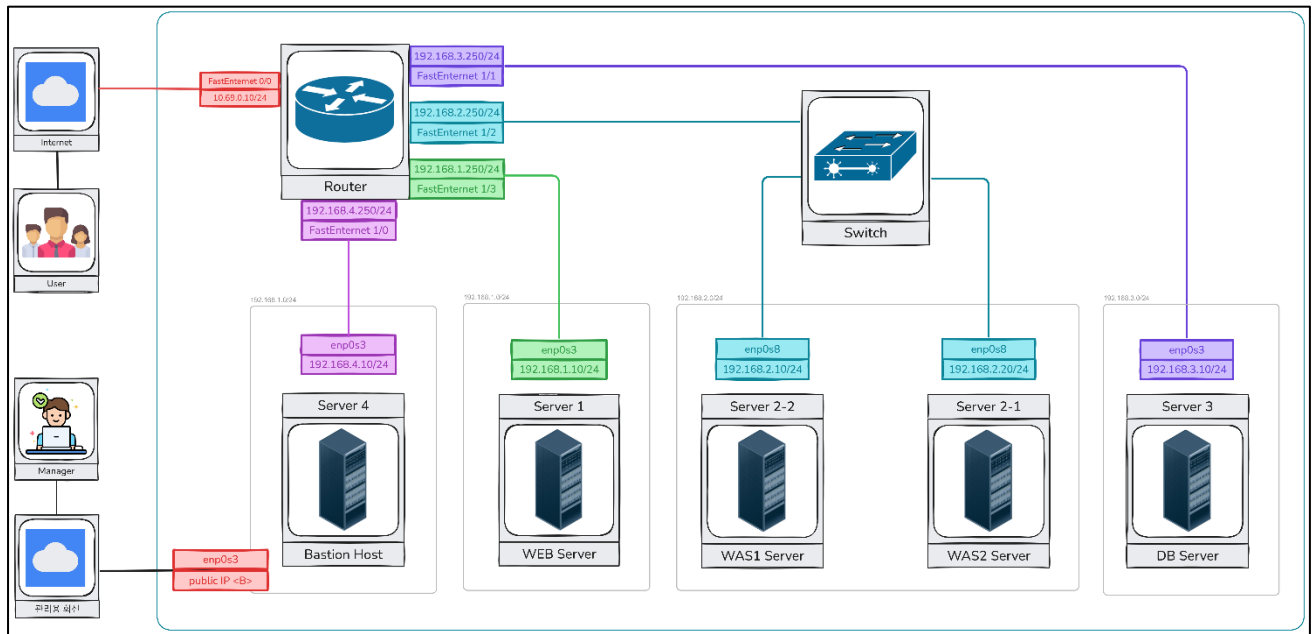
■ 구성 요소

- 서비스용 서버 4대 및 관리용 PC 1대, ISP와 연결된 라우터 1대, 스위치 1대
- 인터넷 회선 2개: 서비스용, 관리자 접근용
- 내부 서브넷 4개 / 서비스 서브넷 3개, 관리용 1개
- 서버용 VM 사양
 - [WEB 서버] CPU: 2Core, RAM: 4GB , 저장공간: 25GB
 - [WAS와 DB 서버] CPU: 2Core, RAM: 2GB , 저장공간: 25GB
- BastionHost/Logging 서버용 VM 사양:
 - CPU: 4Core . RAM: 12GB , 저장공간: 50GB
 - ELK의 최소사양: 4Core, 8GB이상
- 라우터 이미지: C3600
- 포트 확장: 라우터 우클릭으로 설정 진입 → Slots 탭 → Slot1에 NM-16ESW 추가

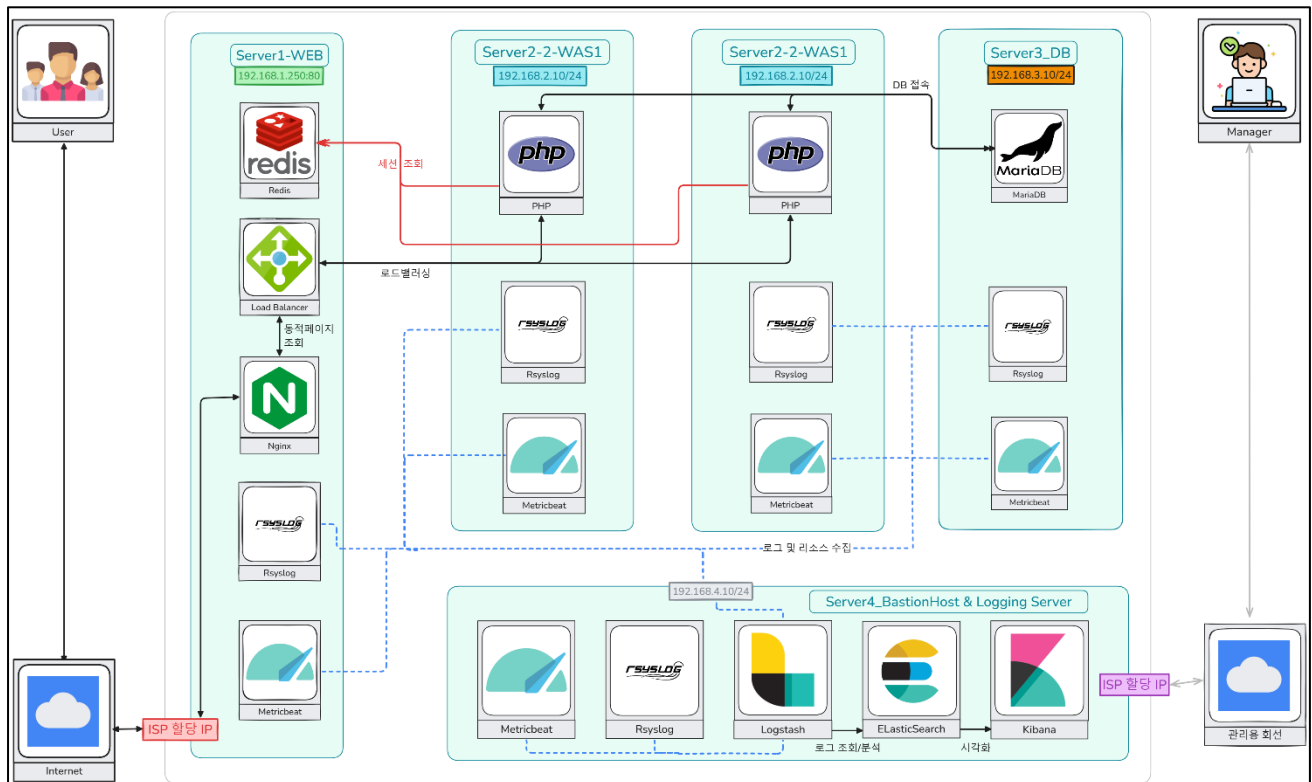
■ 실습 시나리오

- 모놀로지식 3티어 아키텍처를 MSA 3티어 아키텍처로 전환. 그 과정에서 분리된 로그를 하나로 통합하기 위한 로깅서버를 추가로 구축한다.
- 로깅서버는 ELK state를 사용하며, 시용량 리소스체크를 위해 추가 모듈(Metricbeats)을 설치한다

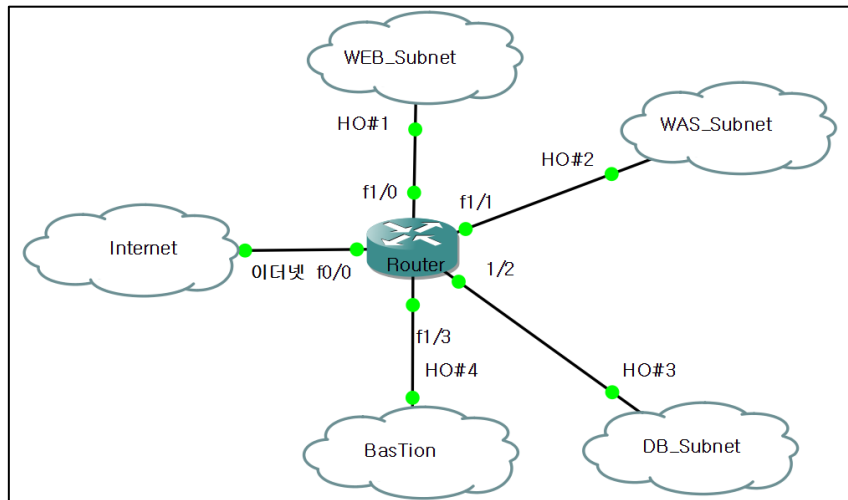
■ 물리적 구성도



■ 논리적 구성도



■ GNS3 연결



■ IP 테이블

구분	장비명	어댑터 번호	IP 주소/CIDR	목적	비고
Server 1	WEB 서버	enp0s3	192.168.1.10 /24	WEB 페이지 제공	Router와 연결, HO#1
Server 2-1	WAS 서버	enp0s3	192.168.2.10 /24	WEB/DB 통신	Router와 연결, HO#2
Server 2-2	WAS 서버	enp0s3	192.168.2.20 /24	WEB/DB 통신	Router와 연결, HO#2
Server 3	DB 서버	enp0s3	192.168.3.10 /24	WEB/DB 통신	Router와 연결, HO#3
Server 4	Bastion Host	enp0s3	192.168.4.10 /24	관리용	Router와 연결, HO#4
		enp0s8	DHCP	외부/인터넷 연결용	실습을 위해 Bridge 연결
Router	Router	FastEthernet 0/0	172.16.7.250/8	인터넷 연결용	Window VM과 연결
		FastEtherne 1/0	192.168.1.250/24	WEB GW	WEB와 연결, HO#1
		FastEtherne 1/1	192.168.2.250/24	WAS GW	WAS와 연결, HO#2
		FastEtherne 1/2	192.168.3.250/24	DB GW	DB와 연결, HO#3
		FastEtherne 1/3	192.168.4.250/24	Bastion Host GW	Bastion과 연결, HO#4
User	Win_U	이더넷	172.16.7.10/24	외부연결 테스트용	Router와 연결, HO#5
Manager	Manager	VM을 실행 중인 REAL_PC 사용			Bastion과 NAT로 연결

■ 추가 사항

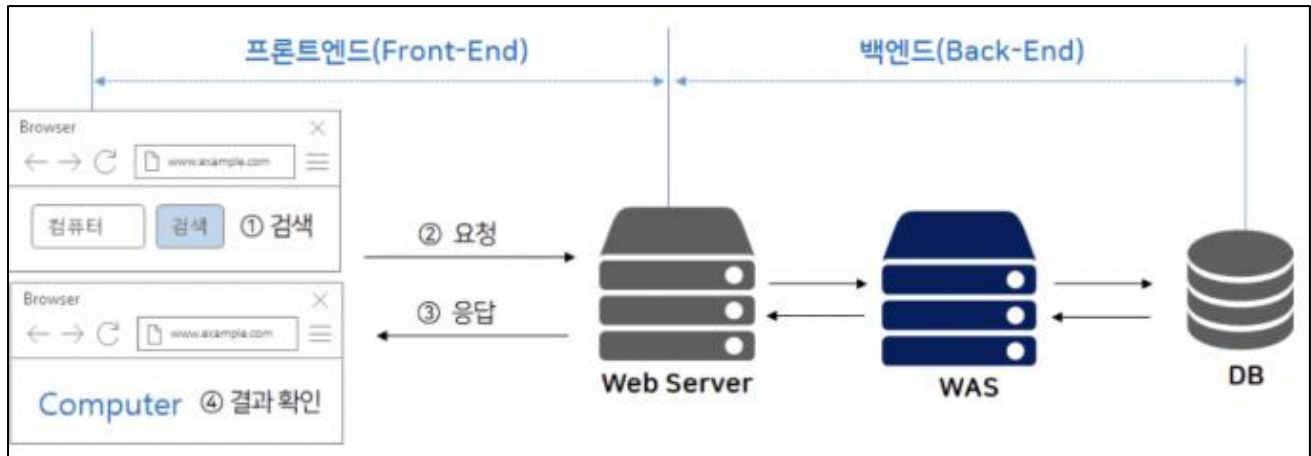
GNS3에서 ROUTER가 Internet과 연결되어 있는 포트에서 DHCP로 IP로 받으지 못하는 제한된 설정이 존재. 이는 VM들이 Router와는 외부 통신이 불가능한 상황. 초반에 패키지 설치를 위해서 VM 어댑터에 NAT 어댑터로 외부와 연결하여 패킷지만 받고 Hostonly 어댑터에 연결.

호스트 전용 어댑터 정보(추가방법: 도구(네트워크) → 만들기)

이름	IPv4 접두사	IPv6 접두사	DHCP 서버
VirtualBox Host-Only Ethernet Adapter	192.168.1.1/24		사용 안 함
VirtualBox Host-Only Ethernet Adapter #2	192.168.2.1/24		사용 안 함
VirtualBox Host-Only Ethernet Adapter #3	192.168.3.1/24		사용 안 함
VirtualBox Host-Only Ethernet Adapter #4	192.168.4.1/24		사용 안 함
VirtualBox Host-Only Ethernet Adapter #5	120.0.0.1/24		사용 안 함

개념 설명

1. 3 티어 아키텍처(3-Tier Architecture)



- 3 티어 아키텍처는 소프트웨어 구조를 사용자 인터페이스를 담당하는 프리젠테이션 계층, 애플리케이션 처리를 수행하는 비즈니스 로직 계층, 데이터를 관리하는 데이터베이스 계층의 세 가지 계층으로 구분하여 관리하는 구조.
- 각 계층이 독립적으로 분리되어 있어 유지보수가 용이하고 확장성이 뛰어나.
- 본 실습에서는 WEB(Nginx), WAS(Apache+PHP), DB(MariaDB) 구조 사용.

2. WEB 서버(WEB Server)

- 사용자의 HTTP 요청을 처리하고 정적 콘텐츠(HTML, CSS, 이미지 등)를 전달하거나, 동적 요청을 WAS 서버로 전달하는 역할을 합니다.
- Nginx 는 성능이 뛰어나고 동시 접속 처리에 효율적이며, 본 실습에서는 로드 밸런서 역할로서 여러 WAS 서버 간 요청을 분산 처리합니다.

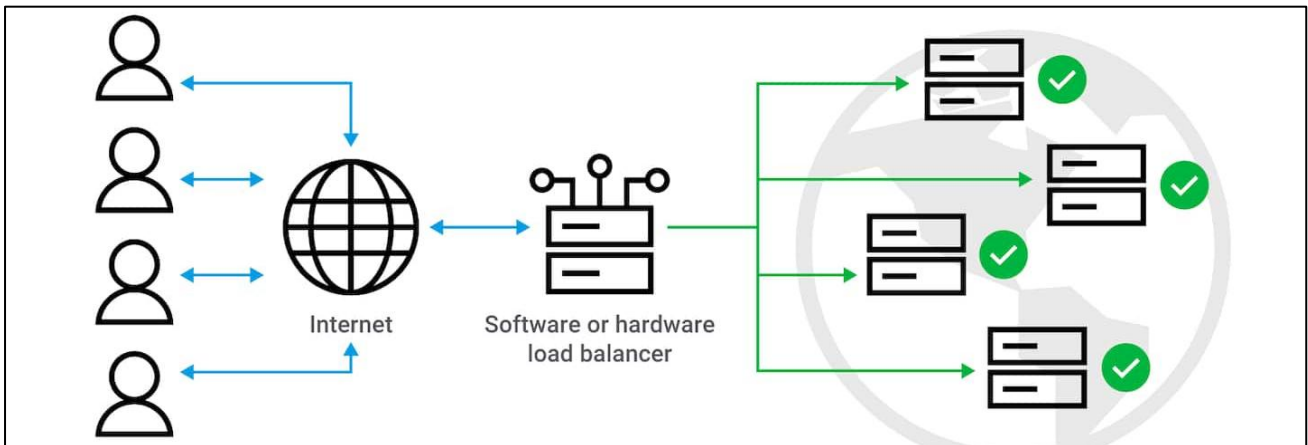
3. WAS 서버(WEB Application Server)

- 동적 콘텐츠를 생성하여 사용자에게 제공합니다.
- Apache 는 웹 애플리케이션 서버로서 PHP 와 연동하여 사용자 요청을 처리합니다. PHP 는 서버 측 스크립트 언어로 DB 접근 및 애플리케이션 로직 구현에 사용됩니다.

4. DB 서버(DataBase Server)

- 데이터를 저장, 관리, 제공하는 역할을 합니다.
- MariaDB 는 MySQL 에서 파생된 오픈 소스 관계형 데이터베이스 시스템(RDBMS)으로 안정적이며 오픈 소스 생태계에서 널리 사용됩니다.

5. 로드밸런싱(Load Balancing)

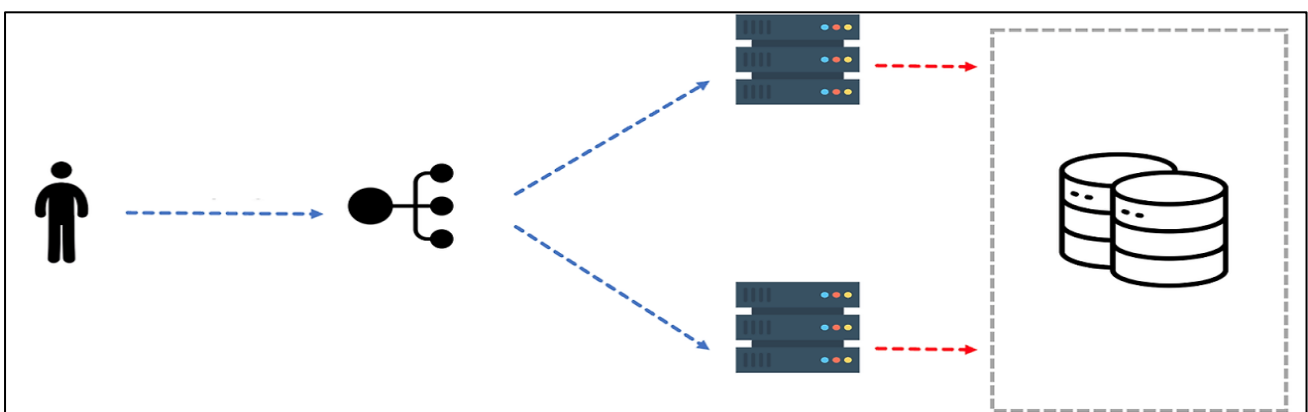


- 여러 서버로 들어오는 요청을 균형 있게 분산 처리하여 서버 부하를 낮추고, 장애 발생 시에도 서비스의 연속성을 보장하는 기술입니다.
- 본 실습에서는 Nginx 를 통해 구현합니다.
- 로드밸런싱 방식으로는 ~~가 있습니다.

6. 세션 관리(Session Management)

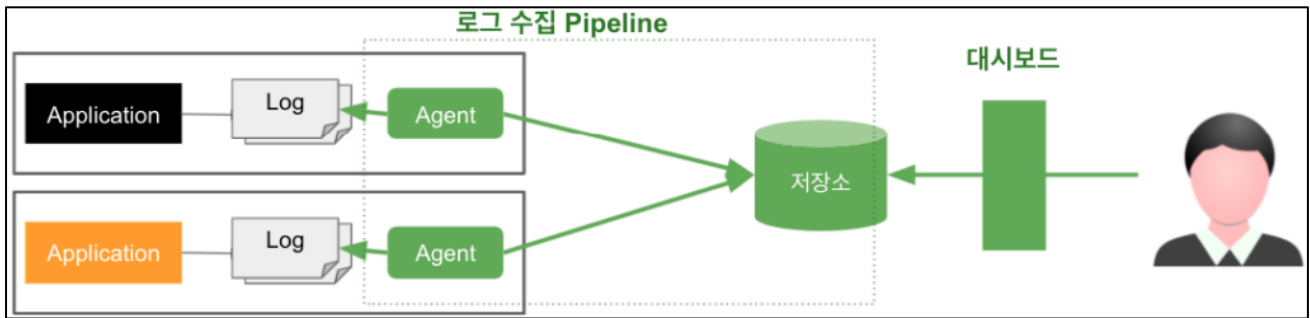
- 사용자의 상태 정보를 서버에서 관리하는 방식입니다
- 서버가 여러 대일 경우 세션 정보를 일관성 있게 유지하기 위해 중앙화된 세션 관리가 필요합니다.
- 본 실습에선 Redis 와 같은 시스템이 이 역할을 수행합니다.

7. Redis



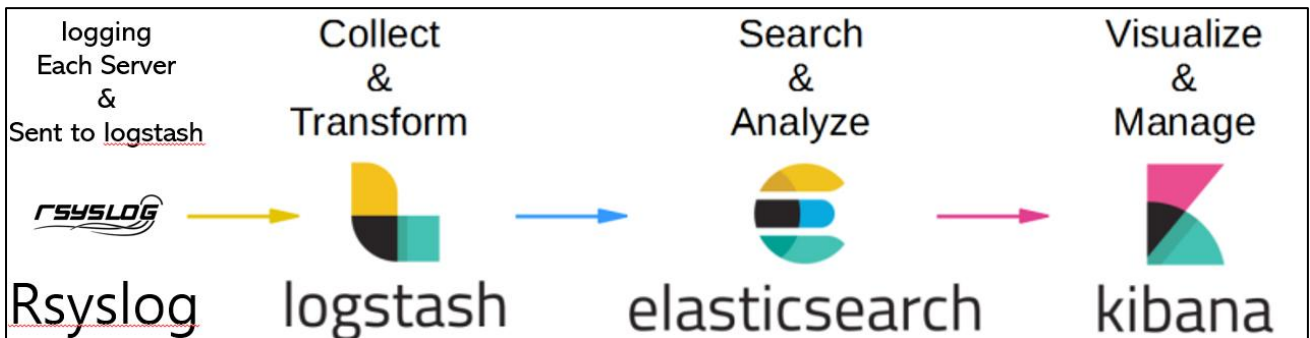
- 인메모리 기반의 키-값 저장소로 빠른 성능이 특징이며, 세션 데이터 관리나 캐싱 목적으로 자주 사용됩니다.
- 본 실습에서는 Redis 를 이용하여 두 대의 WAS 서버 간에 세션을 공유하여 사용자 경험을 일관성 있게 유지합니다.

8. 중앙집중식 로깅(Centralized Logging)



- 다양한 서버에서 발생한 로그를 중앙의 한 지점에 모아 관리 및 분석하는 방식
- 이를 통해 로그 관리의 효율성을 높이고 빠르게 문제를 탐지하고 대응 가능.
- 본 실습에서는 ELK 스택과 Rsyslog 를 활용하여 이 기능을 통해 실현.

9. ELK 스택(Elasticsearch, Logstash, Kibana)



- ELK 는 로그 수집 및 분석을 위한 오픈 소스 플랫폼으로, 로그 데이터의 실시간 수집, 저장, 시각화에 사용됩니다.
 - **Elasticsearch**: 데이터를 빠르게 검색할 수 있는 분산형 검색 엔진입니다.
 - **Logstash**: 로그 데이터를 수집하고 처리하여 Elasticsearch 에 전달합니다.
 - **Kibana**: Elasticsearch 의 데이터를 시각화하여 이해하기 쉽게 제공합니다.
- 다양한 모듈 지원으로 필요에따라 원하는 정보 또한 확인 가능합니다.

10. Rsyslog

- 서버에서 발생하는 로그를 원격 서버로 전송할 수 있는 소프트웨어로, 로그를 중앙에서 효율적으로 관리하기 위해 사용됩니다.

사용 기술 선정 이유

본 실습 시나리오의 목적은 3티어 아키텍처를 구축하여 효율적인 웹 서비스 제공 및 안정적인 시스템 운영을 달성하고, 중앙 집중식 로깅 시스템을 통해 시스템 로그의 효과적인 관리와 신속한 장애 대응 환경을 구축하는 것입니다. 이러한 목적을 달성하기 위해 다양한 관련 기술들을 조사하였으며, 최종적으로 본 실습의 요구사항과 가장 적합한 기술을 선정하기 위해 아래의 과정을 거쳤습니다.

목적에 적합한 기술 후보군을 선정하기 위해 여러 기술을 조사하였고, 이에 따라 WEB 서버, WAS 서버, DB 서버, 중앙 집중식 로깅 시스템 등의 분야에서 널리 사용되는 기술 리스트를 구성하였습니다. 실습 시나리오에 가장 적합한 기술을 최종적으로 선정하였습니다.

조사된 주요 기술 후보군은 다음과 같습니다.

- WEB 서버 후보군: Apache, Nginx
- WAS 서버 후보군: Apache + PHP, Tomcat, Node.js
- DB 서버 후보군: MySQL, MariaDB, PostgreSQL
- 캐싱 및 세션 관리 후보군: Redis, Memcached
- 중앙 집중식 로깅 후보군: ELK Stack (Elasticsearch, Logstash, Kibana), Splunk, Graylog
- 로그 전송 후보군: Rsyslog, Fluentd, Filebeat
- 리소스 모니터링: ELk Metricbeat 모듈

가. WEB 서버: NGINX(LB), Redis

- ➔ **Nginx:** 동시 접속 처리 성능이 우수하고 적은 리소스로 효율적으로 작동. 특히 로드밸런싱 기능이 뛰어나 다수의 WAS 서버 간 트래픽을 균형 있게 분산시켜 서버 안정성을 높임.
- ➔ **Redis:** 빠른 인메모리 데이터 처리 속도로 여러 WAS 서버 간 세션 공유를 효과적으로 처리할 수 있습니다.

나. WAS 서버: Apache + php

- ➔ **Apache:** 널리 사용되는 안정적인 웹 서버로 PHP와의 연동성이 뛰어나 동적 웹 페이지 처리에 최적
- ➔ **PHP:** 데이터베이스 연동에 강점. 구현에 효과적.

다. DB 서버: MariaDB

- ➔ MariaDB는 MySQL과의 높은 호환성과 더 나은 성능 최적화를 제공하여 데이터베이스 성능 및 안정성을 요구하는 본 실습에 적합합니다.

라. ELK state

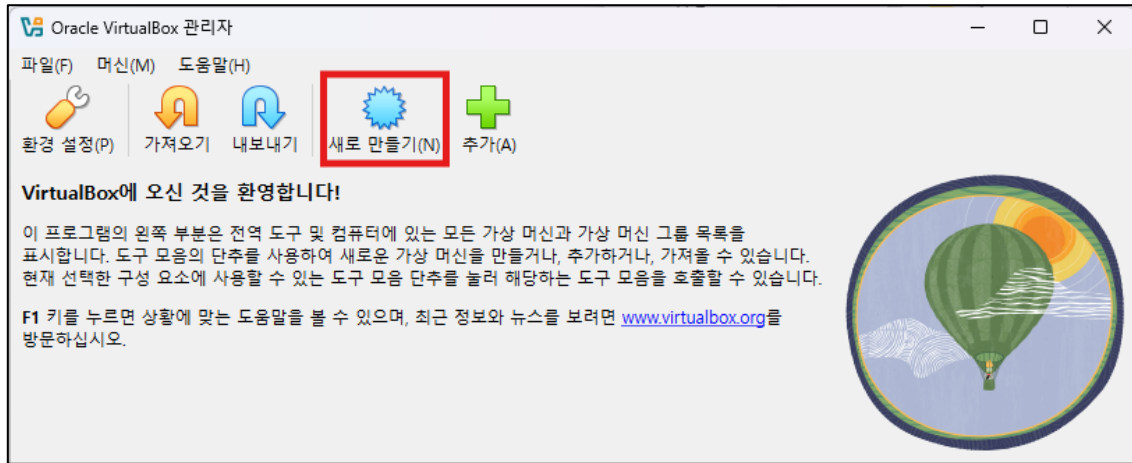
- ➔ **Elasticsearch:** 실시간 데이터 처리 및 빠른 검색 능력으로 대량 로그 데이터 처리에 적합
- ➔ **Logstash:** 다양한 소스의 로그를 수집, 처리, 전송하는 기능과 플러그인 확장성이 뛰어남.
- ➔ **Kibana:** 직관적인 시각화 및 분석 기능으로 로그 데이터를 효율적으로 분석하고 모니터링 가능
- ➔ **Rsyslog:** 로그 데이터를 안정적이고 빠르게 전송할 수 있는 신뢰성이 뛰어난 솔루션으로, 본 실습에서 로그를 효과적으로 중앙으로 집중시켜 관리하기에 적합합니다.

마. Metricbeats

- ➔ Metricbeat는 서버 및 시스템 리소스 데이터를 실시간으로 모니터링할 수 있어 효율적인 시스템 운영 및 문제 발생 시 신속한 대응이 가능합니다.

<전제: 외부연결및 패키지 설치>

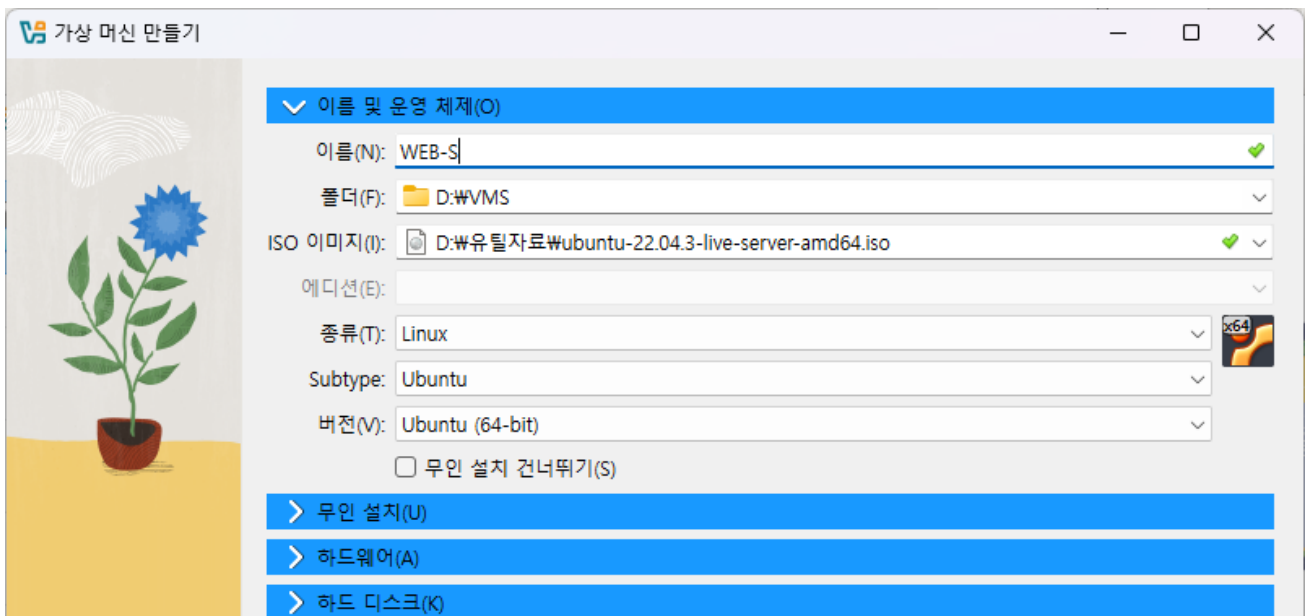
흐름00. 각 VM 생성

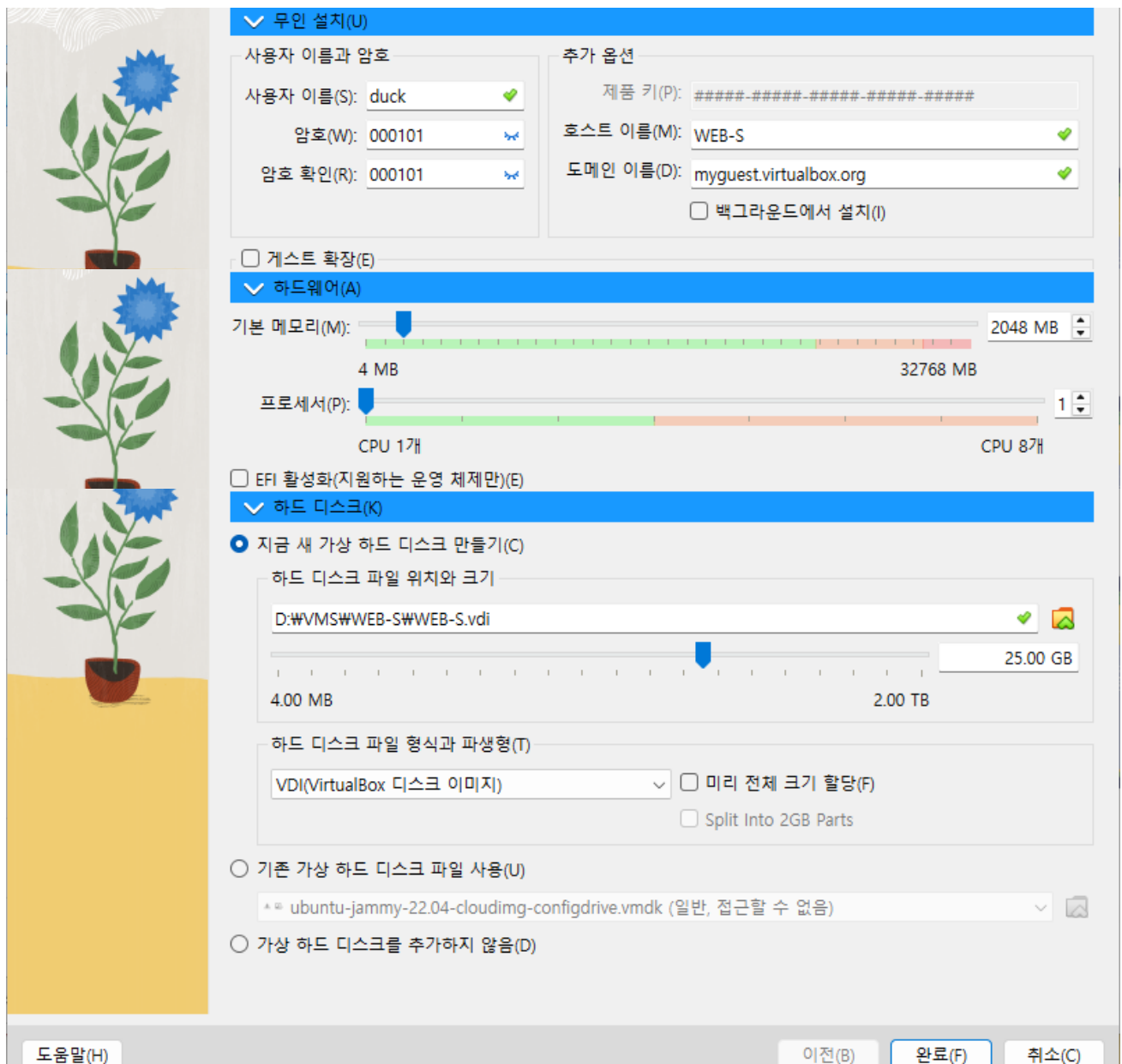


①. 서버 역할용 VM 생성

- WEB

- 이름: Web-S
- 폴더: <사용자 임의> , ISO 이미지: Ubuntu-22.04.3-server-amd64.ISO
- 사용자 이름: duck , 암호: 000101
- 기본 메모리 2048 MB , 프로세서 2

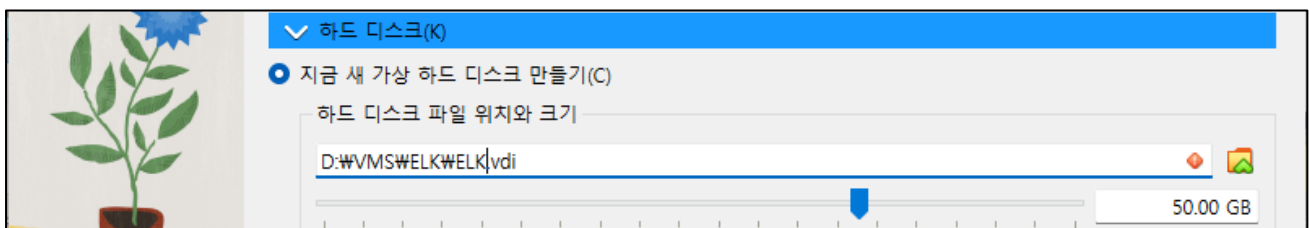




- WAS1 & 2 , DB 서버도 이름만 다르고 전부 동일
 - 각 이름: WAS1-S, WAS2-S DB-S

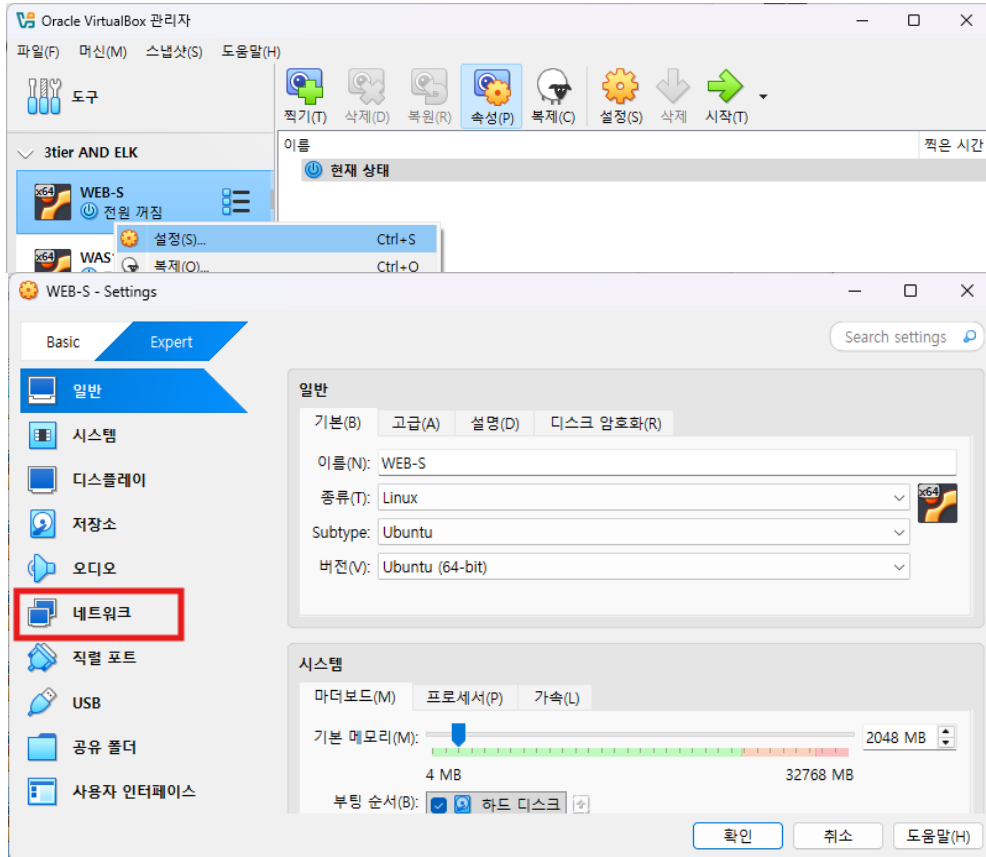
②. Bastion Host VM 생성

- 이름과 하드웨어 성능제외 동일
 - 이름: ELK, CPU: 4Core, 메모리: 12288MB (12GB), 하드디스크 크기: 50GB

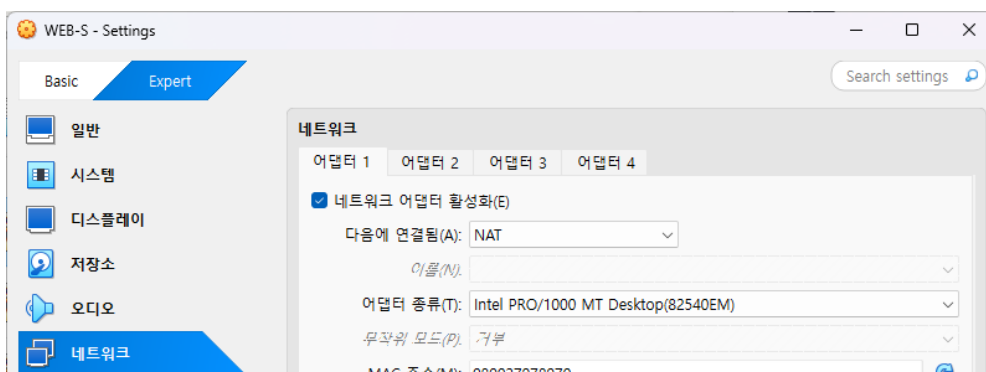


③. 모든 VM을 인터넷에 연결하기

- VM 우클릭 후 설정으로 진입 → 네트워크로 이동



- 다음에 연결된 항목에서 NAT로 변경 후 확인



흐름01. 서버용 VM 에 패키지 설치하기

①. WEB, WAS, DB 서버에 공통적으로 들어가는 패키지 설치

- 다음의 패키지 설치
 - Vim, nano, net-tools, traceroute, tcpdump, cron, metricbeat

- 명령어

```
#저장소추가
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo tee
/usr/share/keyrings/elasticsearch-keyring.asc

echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.asc]
https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee
/etc/apt/sources.list.d/elastic-8.x.list

sudo apt update

sudo apt update
sudo apt install vim nano net-tools traceroute tcpdump cron -y
```

②. WEB용 패키지

- 설치 패키지: Nginx, Redis
- 명령어

```
sudo apt update
sudo apt install vim nano nginx redis-server -y
```

- 상태확인

```
systemctl status nginx
systemctl status redis-server

#버전정보확인
nginx -v
redis-server --version
```

③. WAS용 패키지

- WAS1 WAS2 동일
- 설치 패키지: Apache, php
- 명령어

```
sudo apt update
sudo apt install apache2 php libapache2-mod-php php-mysql php-redis -y
```

- 상태확인

```
sudo systemctl status apache2
```

sudo systemctl status php
#버전정보확인
apache2 -v
php -v
php -m

④. DB용 패키지:

- WAS1 WAS2 동일
- 설치 패키지: Apache, php
- 명령어

sudo apt install mariadb-server -y
● 상태확인
sudo systemctl status mariadb
#버전 정보 확인
mariadb --version

⑤. Bastion Host용 패키지 설치

- 로깅 서버 역할. 더불어 ELK 설치를 위해 저장소 추가해야함
- 설치 패키지: openjdk-17-jdk(Elasticsearch 요구사항), elasticsearch, logstash, kibana, metricbeat
- 명령어

sudo apt install -y openjdk-17-jdk
#ELK 설치
sudo apt install -y elasticsearch logstash kibana metricbeat

- 상태확인

systemctl status elasticsearch
systemctl status logstash
systemctl status kibana
#버전 정보 확인
curl -X GET "localhost:9200"
logstash --version
curl -X GET "localhost:5601/api/status"

흐름02. 각 VM 사설망 전환

①. 외부 연결이 필요한 부분이 끝났으니 사설망으로 전환

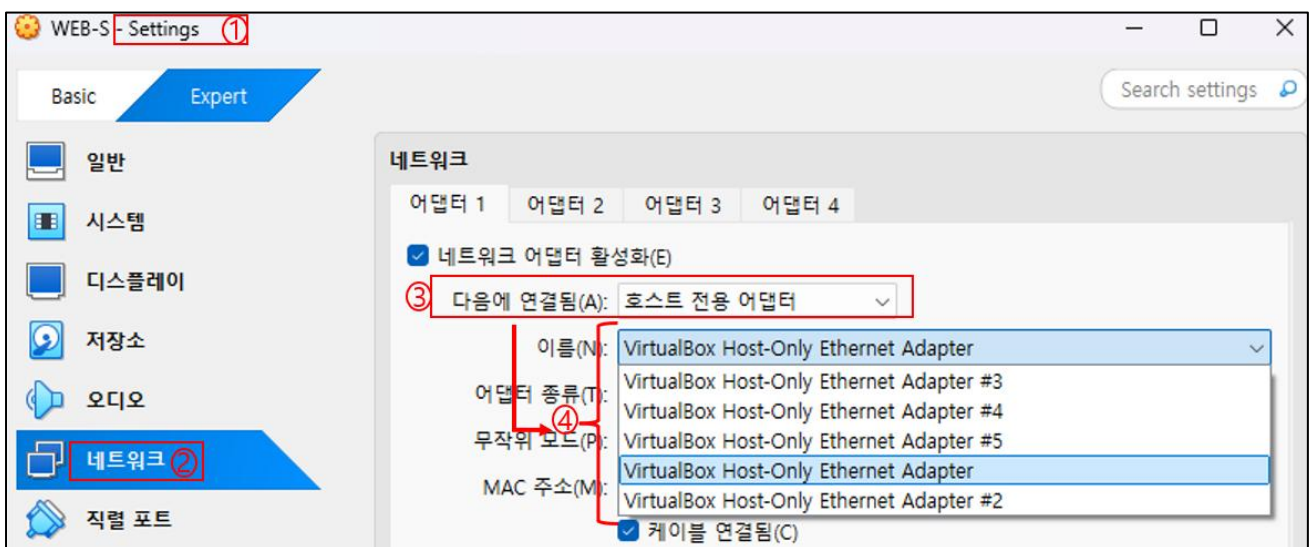
- Host Only 네트워크로 전환

②. 테이블 재확인으로 어떤 어댑터인지 확인

구분	장비명	어댑터 번호	IP 주소/CIDR	목적	비고
Server 1	WEB 서버	enp0s3	192.168.1.10 /24	WEB 페이지 제공	Router와 연결, HO#1
Server 2-1	WAS 서버	enp0s3	192.168.2.10 /24	WEB/DB 통신	Router와 연결, HO#2
Server 2-2	WAS 서버	enp0s3	192.168.2.20 /24	WEB/DB 통신	Router와 연결, HO#2
Server 3	DB 서버	enp0s3	192.168.3.10 /24	WEB/DB 통신	Router와 연결, HO#3
Server 4	Bastion Host	enp0s3	192.168.4.10 /24	관리용	Router와 연결, HO#4
		enp0s8	DHCP	외부/인터넷 연결용	실습을 위해 Bridge 연결

③. VM 설정 → 네트워크 설정 → 다음에 연결됨 →

- 위 표의 비고에 맞게 각 VM 수정



흐름03. 라우터 설정하기

①. 테이블 재확인

구분	장비명	어댑터 번호	IP 주소/CIDR	목적	비고
Router	Router	FastEthernet 0/0	172.16.7.250/8	인터넷 연결용	Window VM과 연결
		FastEthernet 1/0	192.168.1.250/24	WEB GW	WEB와 연결, HO#1
		FastEthernet 1/1	192.168.2.250/24	WAS GW	WAS와 연결, HO#2
		FastEthernet 1/2	192.168.3.250/24	DB GW	DB와 연결, HO#3
		FastEthernet 1/3	192.168.4.250/24	Bastion Host GW	Bastion과 연결, HO#4

②. 각 인터페이스에 IP 부여

- 명령어(설정 모드 → 인터페이스 진입 → IP 부여 → 포트 활성화)

Router# conf t
Router(config)#int fa1/0
Router(config-if)# no sw
Router(config-if)# ip address 192.168.1.250 255.255.255.0
Router(config-if)# no shutdown
Router(config)#int fa1/1
Router(config-if)# no sw
Router(config-if)# ip address 192.168.2.250 255.255.255.0
Router(config-if)# no shutdown
Router(config)#int fa1/2
Router(config-if)# no sw
Router(config-if)# ip address 192.168.3.250 255.255.255.0
Router(config-if)# no shutdown
Router(config)#int fa1/3
Router(config-if)# no sw
Router(config-if)# ip address 192.168.4.250 255.255.255.0
Router(config-if)# no shutdown

- 포트의 IP 및 활성화 확인

```
R1#show ip int b
```

Interface	IP-Address	OK?	Method	Status	Protocol
FastEthernet0/0	unassigned	YES	manual	administratively down	down
FastEthernet0/1	unassigned	YES	NVRAM	administratively down	down
FastEthernet1/0	192.168.1.250	YES	NVRAM	up	up
FastEthernet1/1	192.168.2.250	YES	NVRAM	up	up
FastEthernet1/2	192.168.3.250	YES	NVRAM	up	up
FastEthernet1/3	192.168.4.250	YES	NVRAM	up	up
FastEthernet1/4	unassigned	YES	unset	up	down

3티어 구축하기

< DB 서버 구축 >

흐름04. IP 및 라우팅 설정

①. Netplan 파일 수정을 통한 IP 및 라우팅 변경

- 띄어쓰기 2칸으로 들여쓰기

```
sudo vim /etc/netplan/00-installer-config.yaml

network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.3.10/24
      routes:
        - to: 192.168.3.0/16
          via: 192.168.3.250
```

②. 설정 적용 및 확인

```
sudo netplan apply
```

```
ip address 혹은 ifconfig
```

```
ip route
```

흐름05. MariaDB 설정 변경

①. MariaDB에 root 사용자로 접속

- 명령어

```
sudo mysql -u root
```

②. root 비밀번호 변경 및 외부 접속 허용 계정 생성

- 마리아DB 내부에서 실행

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'root';
FLUSH PRIVILEGES;
```

```

--- 로컬 사용자 계정
CREATE USER 'duck'@'%' IDENTIFIED BY '000101';
GRANT ALL PRIVILEGES ON *.* TO 'duck'@'%' WITH GRANT OPTION;

--- was 접속용 계정
CREATE USER 'was_user'@'%' IDENTIFIED BY 'was_pass';

--- 베스천 접근용 계정
CREATE USER 'bastion_user'@'%' IDENTIFIED BY 'bastion_pass';
GRANT ALL PRIVILEGES ON *.* TO 'bastion_user'@'%' WITH GRANT OPTION;

FLUSH PRIVILEGES;
EXIT;

```

③. 외부 접속 허용하기

- /etc/mysql/mariadb.conf.d/50-server.cnf 파일 수정.
 - bind-address localhost 혹은 bind-address 127.0.0.1을 bind-address 0.0.0.0으로 수정

```
bind-address = 0.0.0.0
```

④. 테스트용 DB와 Table 생성

- MariaDB 진입: `mysql -u root -p`
- 테스트용 DB명: ShopDB

```

CREATE DATABASE ShopDB;
USE ShopDB;

MariaDB [(none)]> CREATE DATABASE Shopdb;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE Shopdb;
Database changed
MariaDB [Shopdb]>

```

- 테스트용 Table명: product

```

CREATE TABLE product (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(255) NOT NULL,
  quantity INT NOT NULL DEFAULT 10,
  image_url VARCHAR(255)
);

```

```

MariaDB [ShopDB]>
MariaDB [ShopDB]>
MariaDB [ShopDB]> CREATE TABLE product (
  ->   product_id INT AUTO INCREMENT PRIMARY KEY,
  ->   product_name VARCHAR(255) NOT NULL,
  ->   quantity INT NOT NULL DEFAULT 10,
  ->   image_url VARCHAR(255)
  -> );
Query OK, 0 rows affected (0.078 sec)

```

```

INSERT INTO product (product_name, quantity, image_url) VALUES
('Awesome Product', 15, '/Logo/awesome_product.png'),
('Cool Gadget', 20, '/Logo/cool_gadget.png'),
('Trendy Item', 5, '/Logo/trendy_item.png');

```

```

MariaDB [ShopDB]> DESC products;

```

Field	Type	Null	Key	Default	Extra
product_id	int(11)	NO	PRI	NULL	auto_increment
category_id	int(11)	YES	MUL	NULL	
product_name	varchar(255)	NO		NULL	
description	text	YES		NULL	
price	decimal(10,2)	NO		NULL	
stock	int(11)	NO		0	
image_url	varchar(255)	YES		NULL	
created_at	datetime	YES		current_timestamp()	

```

8 rows in set (0.001 sec)

```

● 테이블 스키마 삽입

```

INSERT INTO product (product_name, quantity, image_url) VALUES
('Awesome Product', 15, '/Logo/awesome_product.png'),
('Cool Gadget', 20, '/Logo/cool_gadget.png'),
('Trendy Item', 5, '/Logo/trendy_item.png');

```

```

MariaDB [ShopDB]> INSERT INTO product (product_name, quantity, image_url) VALUES
  -> ('Awesome Product', 15, '/Logo/awesome_product.png'),
  -> ('Cool Gadget', 20, '/Logo/cool_gadget.png'),
  -> ('Trendy Item', 5, '/Logo/trendy_item.png');
Query OK, 3 rows affected (0.049 sec)
Records: 3  Duplicates: 0  Warnings: 0

MariaDB [ShopDB]> █

```

< WEB 서버 및 Redis 설정 >

흐름06. IP 및 라우팅 설정

- ①. Netplan 파일로 진입하여 고정 IP 및 라우팅 부여
 - netplan 설정 파일 위치: /etc/netplan/00-installer-config.yaml
- ②. IP 테이블 재확인. IP 및 CIDR

구분	장비명	어댑터 번호	IP 주소/CIDR	목적	비고
Server 1	WEB 서버	enp0s3	192.168.1.10 /24	WEB 페이지 제공	Router와 연결, HO#1
Server 2-1	WAS 서버	enp0s3	192.168.2.10 /24	WEB/DB 통신	Router와 연결, HO#2
Server 2-2	WAS 서버	enp0s3	192.168.2.20 /24	WEB/DB 통신	Router와 연결, HO#2
Server 3	DB 서버	enp0s3	192.168.3.10 /24	WEB/DB 통신	Router와 연결, HO#3
Server 4	Bastion Host	enp0s3	192.168.4.10 /24	관리용	Router와 연결, HO#4
		enp0s8	DHCP	외부/인터넷 연결용	실습을 위해 Bridge 연결

③. netplan 파일 수정

- sudo vim /etc/netplan/00-installer-config.yaml || i키 누르고 입력모드 진입 → 내용 입력 → ESC 키 → :wq 입력(저장 및 나가기)

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.4.10/24
      routes:
        - to: 0.0.0.0/0
          via: 192.168.4.250
```

```
network:
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.4.10/24
      routes:
        - to: 192.168.0.0/16
          via: 192.168.4.250
    enp0s8:
      dhcp4: true
  version: 2
```

④. netplan 적용 및 확인

- sudo netplan apply
- ip address 및 ip route show

흐름07. WEB 엔진 Nginx 설정

①. 정적 페이지 생성

- Nginx의 웹 페이지 파일은 /var/www/html/ 밑에 존재
- 페이지 생성 및 내용 작성

```
sudo vim /var/www/html/index.html

<!DOCTYPE html>
<html>
<head> <meta charset="utf-8"> <title>Nginx Index</title> </head>
<body>
    <h1>Nginx index page</h1>
    <p><a href="/img_shop.php">Go to WAS img_shop Page</a> </p>
</body>
</html>

<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Nginx Index</title>
</head>
<body>
    <h1>Nginx index page</h1>
    <p><a href="/img_shop.php">Go to img_shop page</a></p>
</body>
</html>
```

②. 로드밸런싱 설정.

- Nginx 설정 파일 위치: /etc/nginx/nginx.conf → 건들필요 없음
- 추가적 설정 파일: /etc/nginx/conf.d/3tier.conf
➔ WAS가 이중화 된 환경에서 요청을 분산하기 위한 작업

```
sudo vim /etc/nginx/conf.d/3tier.conf

# WAS 백엔드 서버 그룹 정의
upstream was_backend {
    server 192.168.2.10;
    server 192.168.2.20;
}
```

```

server {
    listen 80;
    server_name 192.168.1.10; # 또는 도메인

    # 정적 파일 제공 (HTML, CSS, JS)
    location / {
        root /var/www/html; # WEB 서버에 있는 정적 파일 위치
        index index.html;
    }

    # API 요청은 WAS 서버로 프록시
    location /api/ {
        proxy_pass http://was_backend/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

③. 설정 파일 오류 점검 및 설정 반영

- 파일 권한 확인 및 문법 검사

<pre> sudo chown -R www-data:www-data /var/www/html sudo chmod -R 755 /var/www/html </pre>
<pre> sudo nginx -t </pre>

* OK가 뜨면 통과

- 재시작을 통한 설정 반영

<pre> sudo systemctl restart nginx </pre>
<pre> sudo systemctl status nginx </pre>

흐름08. Redis DB 설정

①. 외부 접속 허용 및 비밀번호 설정

- /etc/redis/redis.conf 파일 편집

- 외부 접속 허용: bind 127.0.0.1 → bind 0.0.0.0
- 비밀번호 추가: requirepass [비밀번호]
-

- 파일 변경

sudo vim /etc/redis/redis.conf
bind 0.0.0.0 requirepass duck
대략 68번째 줄에 존재 <pre>##### NETWORK ##### # By default, if no "bind" configuration directive is specified, Redis listens # for connections from all available network interfaces on the host machine. # It is possible to listen to just one or multiple selected interfaces using # the "bind" configuration directive, followed by one or more IP addresses. # # Examples: # # bind 192.168.1.100 10.0.0.1 # bind 127.0.0.1 ::1 # ~~~~~ bind 0.0.0.0 requirepass duck</pre>

②. 재시작을 통한 설정 반영

- 재시작 및 확인

sudo systemctl restart redis-server
sudo systemctl status redis-server

③. 암호 로그인 동작 확인

- 미인증시 키 조회: **redis-cli** 로 진입

```
vagrant@SERVER-1-WEB:~$ redis-cli
127.0.0.1:6379> ping
(error) NOAUTH Authentication required.
127.0.0.1:6379> keys *
(error) NOAUTH Authentication required.
127.0.0.1:6379> █
```

- 인증 후 키 조회

→ auth [비밀번호]를 통한 인증

```
127.0.0.1:6379> auth duck
OK
127.0.0.1:6379> ping
PONG
127.0.0.1:6379> keys *
1) "PHPREDIS_SESSION:te8r98k4v2nkg8kt7l5tjgmchr"
2) "PHPREDIS_SESSION:c523m4i4k79log8c51qn56tt7b"
3) "PHPREDIS_SESSION:6evus0j17b8uomockm5hicm1bb"
```

< WAS 설정 및 이중화/세션유지 >

흐름09. IP 및 라우팅 설정

①. Netplan 파일로 진입하여 고정 IP 및 라우팅 부여

- netplan 설정 파일 위치: /etc/netplan/00-installer-config.yaml

②. IP 테이블 재확인. IP 및 CIDR

구분	장비명	어댑터 번호	IP 주소/CIDR	목적	비고
Server 1	WEB 서버	enp0s3	192.168.1.10 /24	WEB 페이지 제공	Router와 연결, HO#1
Server 2-1	WAS 서버	enp0s3	192.168.2.10 /24	WEB/DB 통신	Router와 연결, HO#2
Server 2-2	WAS 서버	enp0s3	192.168.2.20 /24	WEB/DB 통신	Router와 연결, HO#3
Server 3	DB 서버	enp0s3	192.168.3.10 /24	WEB/DB 통신	Router와 연결, HO#4
Server 4	Bastion Host	enp0s3	192.168.4.10 /24	관리용	Router와 연결, HO#
		enp0s8	DHCP	외부/인터넷 연결용	실습을 위해 Bridge 연결

③. netplan 파일 수정

- sudo vim /etc/netplan/00-installer-config.yaml WAS1은 2.10 WAS2는 2.20

<pre>network: version: 2 ethernets: enp0s3: dhcp4: false addresses: - 192.168.2.10/24 routes: - to: 0.0.0.0/0 via: 192.168.2.250</pre>	<pre>network: version: 2 renderer: networkd ethernets: enp0s8: addresses: - 192.168.2.10/24 routes: - to: 0.0.0.0/0 via: 192.168.2.250 enp0s9: addresses: - 120.0.0.22/24</pre>	<pre>network: version: 2 renderer: networkd ethernets: enp0s8: addresses: - 192.168.2.20/24 routes: - to: 0.0.0.0/0 via: 192.168.2.250 enp0s9: addresses: - 120.0.0.22/24</pre>
--	---	---

④. netplan 적용 및 확인

- sudo netplan apply → ip address 및 ip route show

흐름10. Apache 기본 사이트 설정

- ①. 설정 파일 위치: /etc/apache2/sites-available/000-default.conf
- ②. 아래와 같이 수정. WAS1/WAS2에 따라 **ServerName** 항목 IP 수정

```
sudo vim /etc/apache2/sites-available/000-default.conf
```

```
<VirtualHost *:80>
    ServerName 192.168.2.10
    DocumentRoot /var/www/html

    <Directory /var/www/html>
        Options Indexes FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

```
<VirtualHost *:80>
    #ServerAdmin webmaster@localhost
    #DocumentRoot /var/www/html

    ServerName 192.168.2.10
    DocumentRoot /var/www/html

    <Directory /var/www/html>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

- ③. Apache 재시작으로 설정 적용

```
sudo systemctl restart apache2
```

```
sudo systemctl status apache2
```

흐름11. Redis 세션 설정 (WAS1, WAS2 모두)

- ①. 세션유지를 위하여 PHP를 이용한 redisDB 연동
 - 설정파일 위치: /etc/php/8.1/apache2/php.ini

②. 설정 파일 변경. [Session] 항목 밑에 다음과 같이 수정/추가

```
session.save_handler = redis
session.save_path = "tcp://192.168.1.10:6379?auth=duck"
```

```
[Session]
; Handler used to store/retrieve data.
; https://php.net/session.save-handler
;session.save_handler = files
session.save_handler = redis

; Argument passed to save_handler. In the case of files, this is the path
; where data files are stored. Note: Windows users have to change this
; variable in order to use PHP's session functions.
;
; The path can be defined as:
;
;     session.save_path = "N;/path"
session.save_path = "tcp://192.168.1.10:6379?auth=duck"
```

③. 재시작을 통한 설정 반영

```
sudo systemctl restart apache2
```

④. 테스트용 파일 배포

- 아피치 기본 index파일 위치: /var/www/html/
- 테스트 파일(img_shop) 생성 및 내용 추가

```
#WEB 서버
sudo vim /var/www/html/img_shop.html

<!-- img_shop.html 파일은 문서 마지막에 첨부 -->

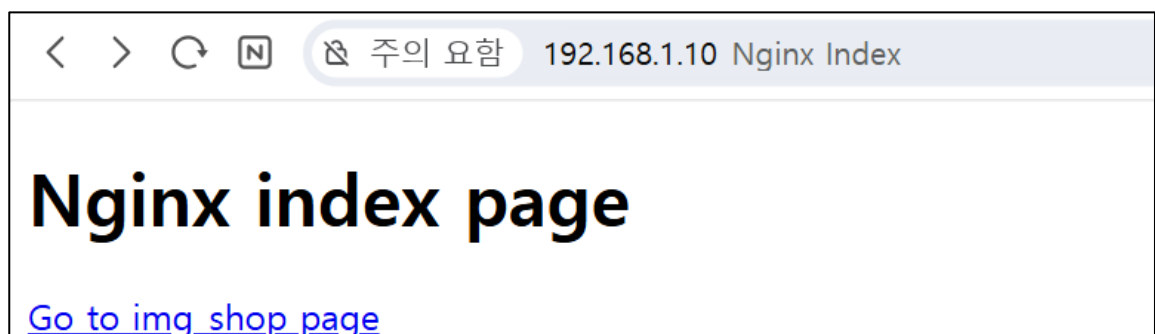
#WAS 서버
sudo vim /var/www/html/api2.php

<!-- api2.php 파일은 문서 마지막에 첨부 -->
```

흐름12. 테스트

①. WEB 서버로 진입 : 192.168.1.10

- 브라우저를 열어 주소창에 192.168.1.10 입력 후 Enter키 입력



②. Go to Img_Shop 클릭

The screenshot shows a web browser at the URL `192.168.1.10 - 구매 페이지`. The page content includes a yellow button labeled **바로 구매하기**, a text field labeled **남은 수량:**, and a **방문 횟수: 1** (where '1' is circled in red). Below these are the **Session ID: b50s656efnou9q8nn43o9k95bc** and **컨테이너 식별자: WAS1-S**. The developer tools on the right show the HTML structure, with the `<div class="product-page">` element selected. The console shows no errors.

③. 새로고침을 통한 다른 WAS 페이지로 이동

The screenshot shows the same web browser after a refresh. The page content is identical to the previous one, but the **방문 횟수: 2** (where '2' is circled in red) and **컨테이너 식별자: WAS2-S** have changed. The developer tools on the right show the HTML structure, with the `<title> - 구매 페이지</title>` element selected. The console shows no errors.

- 세션은 동일하지만, 서버 식별자가 바뀐 것을 확인할 수 있음
- 더불어 DB 또한 표시

④. 레디스 DB확인

- WEB-S로 돌아와 레디스 진입 및 비밀번호 입력. 세션 확인

```
redis-cli -h 127.0.0.1 -p 6379
vagrant@SERVER-1-WEB:~$ redis-cli -h 127.0.0.1 -p 6379
127.0.0.1:6379>
AUTH duck
127.0.0.1:6379> AUTH duck
OK
keys *
127.0.0.1:6379> keys *
1) "PHPREDIS_SESSION:b50s656efnou9q8nn43o9k95bc"
```

<!-- 다음장 계속 --->

< 관리용 BastionHost/ELK 서버 설정 >

흐름13. IP 및 라우팅 설정

- ①. BastionHost는 하는 Host Only 어댑터, 다른 하나는 Bridge 어댑터.
 - Hostonly는 고정 IP 부여. Bridge는 DHCP로 받아온다
- ②. Netplan 파일 수정하기. /etc/netplan/00-00-installer-config.yaml

```
sudo vim /etc/netplan/00-00-installer-config.yaml

# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.4.10/24
      routes:
        - to: 192.168.0.0/16
          via: 192.168.4.250
    enp0s8:
      dhcp4: true

  version: 2
```

```
# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.4.10/24
      routes:
        - to: 192.168.0.0/16
          via: 192.168.4.250
    enp0s8:
      dhcp4: true

  version: 2
```

- ③. 설정 적용

```
sudo netplan apply
```

흐름14. Elasticsearch 설정

①. 설치 확인

```
/usr/share/elasticsearch/bin/elasticsearch --version
```

```
duck@ELK:~$ sudo /usr/share/elasticsearch/bin/elasticsearch --version
Version: 8.17.3, Build: deb/a091390de485bd4b127884f7e565c0cad59b10d2/2025-02-28T10:
89129809Z, JVM: 23
```

②. 설정 파일 수정

- 설정 파일 위치: /etc/elasticsearch/elasticsearch.yml
- 수정 사항: 외부 접속 허용, 클러스터 이름 지정, 노드명 지정,

```
network.host: 0.0.0.0
cluster.name: my-elk-cluster
node.name: BastionHost
http.host:0.0.0.0
```

- 파일 수정

```
sudo vim /etc/elasticsearch/elasticsearch.yml
```

```
# ----- Cluster -----
#
# Use a descriptive name for your cluster:
#
cluster.name: my-elk-cluster
#
# ----- Node -----
#
# Use a descriptive name for the node:
#
node.name: BastionHost
#
# ----- Network -----
#
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
#
network.host: 0.0.0.0
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
http.port: 9200
#
# For more information, consult the network module documentation.
#
# Allow HTTP API connections from anywhere
# Connections are encrypted and require user authentication
http.host: 0.0.0.0
```


③. 설정 적용

```
sudo systemctl restart elasticsearch
sudo systemctl status elasticsearch
```

```
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/lib/systemd/system/elasticsearch.service; enabled; vendor preset: en
   Active: active (running) since Sun 2025-03-16 09:20:23 UTC; 36min ago
     Docs: https://www.elastic.co
    Main PID: 708 (java)
      Tasks: 106 (limit: 14217)
     Memory: 6.7G
        CPU: 3min 4.378s
    CGroup: /system.slice/elasticsearch.service
            └─ 708 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC
               1584 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=6
               1622 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/

Mar 16 09:18:14 ELK systemd[1]: Starting Elasticsearch...
Mar 16 09:19:14 ELK systemd-entrypoint[1584]: CompileCommand: dontinline java/lang/invoke
Mar 16 09:19:14 ELK systemd-entrypoint[1584]: CompileCommand: dontinline java/lang/invoke
Mar 16 09:20:23 ELK systemd[1]: Started Elasticsearch.
lines 1-17/17 (END)
```

흐름15. Logstash 설정 (로그 수집)

①. Logstash 설치 확인

```
/usr/share/logstash/bin/logstash --version
```

```
duck@ELK:~$ /usr/share/logstash/bin/logstash --version
Using bundled JDK: /usr/share/logstash/jdk
logstash 8.17.3
```

②. Logstash 기본 설정

- 파일 권한 설정
- 설정 파일 위치: /etc/logstash/

```
sudo chown -R logstash:logstash /etc/logstash
sudo find /etc/logstash -type d -exec chmod 750 {} \;
sudo find /etc/logstash -type f -exec chmod 640 {} \;
```

- pipelines.yml 파일 생성 및 설정

```
sudo mkdir -p /usr/share/logstash/config
sudo vim /usr/share/logstash/config/pipelines.yml
```

```
- pipeline.id: main
  path.config: "/etc/logstash/conf.d/*.conf"
```

```
sudo chown -R logstash:logstash /usr/share/logstash/config
sudo chmod 640 /usr/share/logstash/config/pipelines.yml
```

③. 테스트기본 원격 접속용 syslog 파일 생성

- 설정 파일 위치: /etc/logstash/conf.d/remote-syslog.conf

● 설정 사항:

- Input: 5140번 포트에서 syslog형 로그 수신, 입력된 로그의 타입을 remote_syslog로 지정하여 필터링
- filter: grok 플러그인을 사용하여 syslog 메시지를 구조화된 데이터로 변환. 로그에서 특정부분을 추출 Elasticsearch가 분석 가능하도록 설정
- output: Elasticsearch로 로그를 전송하고, 날짜별로 새로운 인덱스를 생성. Logstash 실행 시 콘솔에 로그를 출력

● 파일 수정

sudo vim /etc/logstash/conf.d/remote-syslog.conf
<pre>input { syslog { port => 5140 type => "remote_syslog" } } filter { grok { match => { "message" => "%{SYSLOGTIMESTAMP:timestamp} %{SYSLOGHOST:hostname} %{DATA:program}: %{GREEDYDATA:log}" } } } output { elasticsearch { hosts => ["http://localhost:9200"] index => "remote-syslog-%{+YYYY.MM.dd}" } stdout { codec => rubydebug } }</pre>

④. 설정 반영

sudo systemctl restart logstash
sudo systemctl status logstash

흐름16. Kibana 설정

①. 설치 확인

```
/usr/share/kibana/bin/kibana --version  
duck@ELK:~$ /usr/share/kibana/bin/kibana --version 2>/dev/null  
{ "log.level": "info", "@timestamp": "2025-03-16T10:18:39.988Z", "log.logger": "elastic-apm-node",  
  "ecs.version": "8.10.0", "agentVersion": "4.10.0", "env": { "pid": 2508, "proctitle": "/usr/share/  
kibana/bin/./node/glibc-217/bin/node", "os": "linux 5.15.0-134-generic", "arch": "x64", "host":  
  "ELK", "timezone": "UTC+00", "runtime": "Node.js v20.18.2" }, "config": { "active": { "source": "st  
art", "value": true }, "breakdownMetrics": { "source": "start", "value": false }, "captureBody": { "sou  
rce": "start", "value": "off", "commonName": "capture_body" }, "captureHeaders": { "source": "start",  
  "value": false }, "centralConfig": { "source": "start", "value": false }, "contextPropagationOnly":  
  { "source": "start", "value": true }, "environment": { "source": "start", "value": "production" }, "glo  
balLabels": { "source": "start", "value": [ "git_rev", "faabb4e47ac99b6f367713ef845613b7313914b8  
  " ] }, "sourceValue": { "git_rev": "faabb4e47ac99b6f367713ef845613b7313914b8" }, "logLevel": { "sou  
rce": "default", "value": "info", "commonName": "log_level" }, "metricsInterval": { "source": "start",  
  "value": 120, "sourceValue": "120s" }, "serverUrl": { "source": "start", "value": "https://kibana-  
cloud-apm.apm.us-east-1.aws.found.io/", "commonName": "server_url" }, "transactionSampleRate":  
  { "source": "start", "value": 0.1, "commonName": "transaction_sample_rate" }, "captureSpanStackTra  
ces": { "source": "start", "sourceValue": false }, "secretToken": { "source": "start", "value": "[REDA  
CTED]", "commonName": "secret_token" }, "serviceName": { "source": "start", "value": "kibana", "comm  
onName": "service_name" }, "serviceVersion": { "source": "start", "value": "8.17.3", "commonName": "  
service_version" }, "activationMethod": "require", "message": "Elastic APM Node.js Agent v4.10  
8.17.3" }
```

②. Kibana 설정

- 설정 파일 위치: /etc/kibana/kibana.yml
- 수정 사항: 외부접속 허용, 데이터를 가져올 Elasticsearch 서버 주소 설정
- 파일 수정

```
sudo vim /etc/kibana/kibana.yml  
server.host: "0.0.0.0"  
elasticsearch.hosts: ["http://localhost:9200"]  
# ===== System: Kibana Server =====  
# Kibana is served by a back end server. This setting specifies the port to use.  
#server.port: 5601  
  
# Specifies the address to which the Kibana server will bind. IP addresses and host names  
# are both valid values.  
# The default is 'localhost', which usually means remote machines will not be able to conn  
# ect.  
# To allow connections from remote users, set this parameter to a non-loopback address.  
server.host: 0.0.0.0  
  
# ===== System: Elasticsearch =====  
# The URLs of the Elasticsearch instances to use for all your queries.  
elasticsearch.hosts: ["http://localhost:9200"]
```

③. 설정 반영

```
sudo systemctl restart kibana  
sudo systemctl status kibana
```

흐름17. 클라이언트 서버에서 Rsyslog 설정

①. 설치 확인

```
rsyslogd -v  
duck@ELK:~$ rsyslogd -v  
rsyslogd 8.2112.0 (aka 2021.12) compiled with:
```

②. 설정 파일 수정

- 설정파일 위치: `sudo vim /etc/rsyslog.d/50-remote.conf`
- 수정 사항: 로그를 ELK가 있는 서버로 전송
- 파일 수정

```
*.* @192.168.4.10:5140
```

③. 설정 반영

```
sudo systemctl restart rsyslog
```

< ELK 사용해보기 >

흐름18. Kibana 접속하기

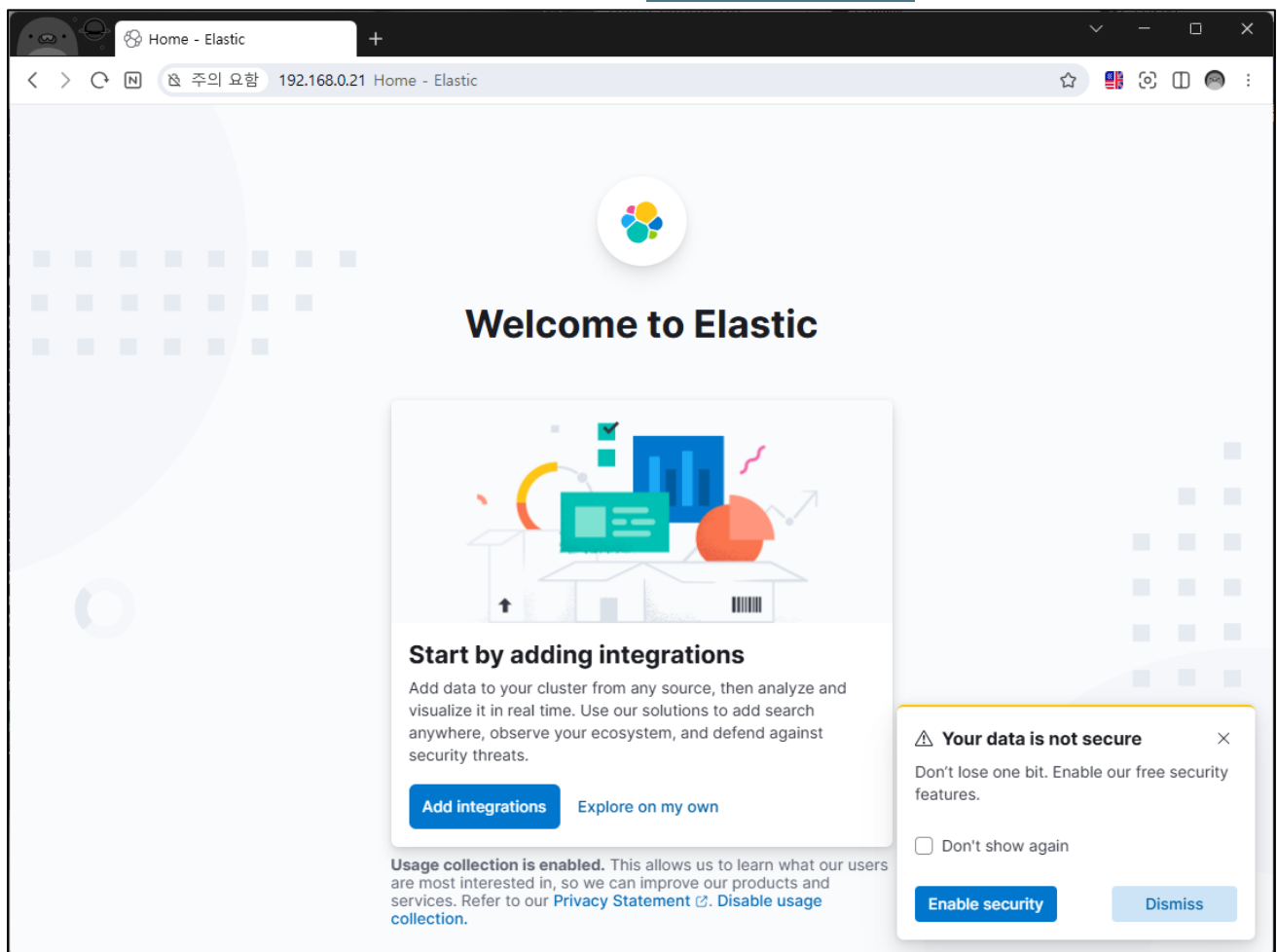
①. 현재 Kibana는 BastionHost에서 실행 중이며 5601 포트에 오픈

- ss -ntpl 명령어를 통해 5601 포트가 열려있는 지 확인

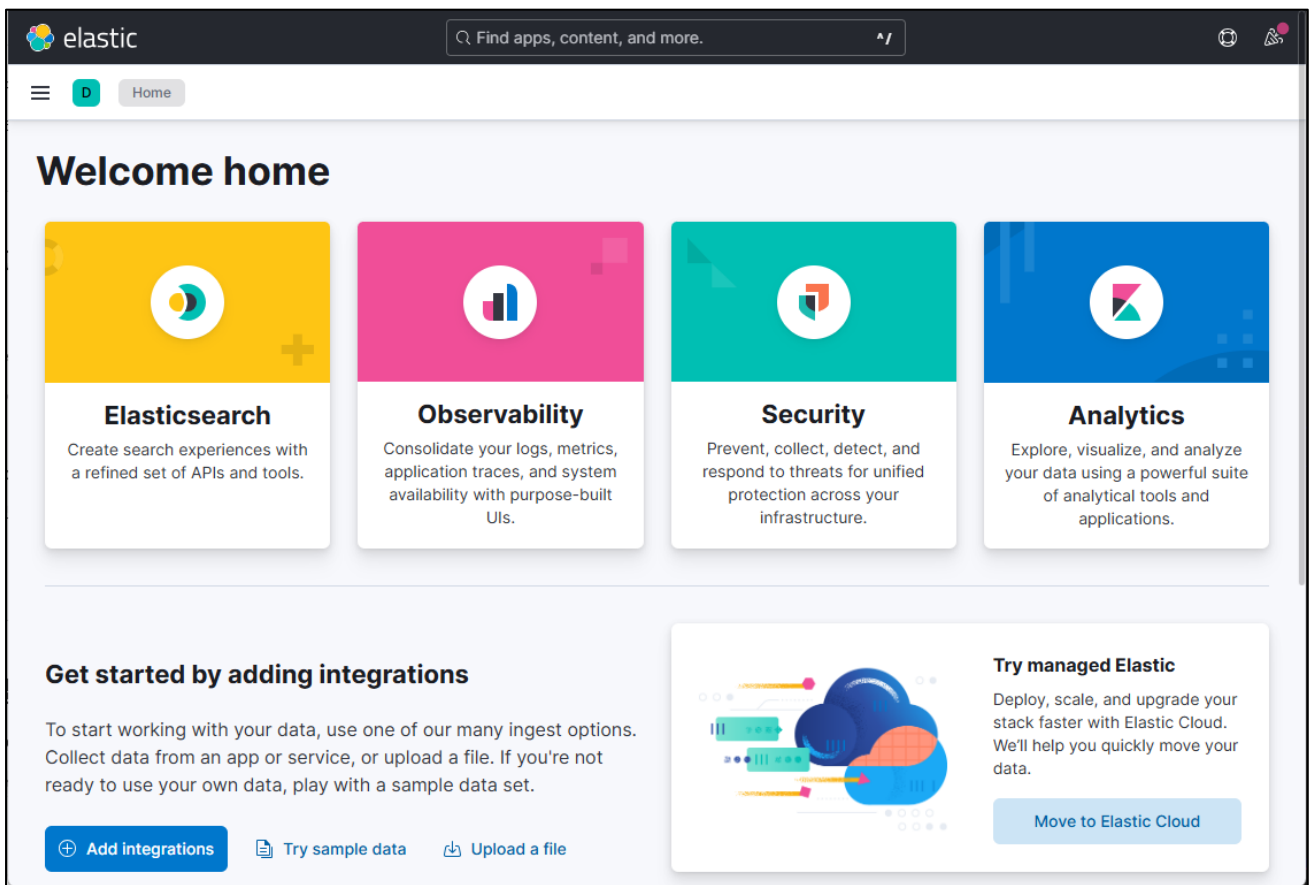
State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	511	0.0.0.0:5601	0.0.0.0:*	
LISTEN	0	4096	127.0.0.1:53	0.0.0.0:*	
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
LISTEN	0	128	127.0.0.1:6010	0.0.0.0:*	
LISTEN	0	50	:::ffff:127.0.0.1:9600	:::ffff:0.0.0.0:*	

②. 베스천에서 연결된 외부 IP 확인하여 브라우저로 진입

- ifconfig 혹은 ip addresses를 통하여 IP 확인
- 확인된 IP에 포트번호 붙혀 웹 브라우저 주소창에 입력 후 진입
- [BastionHost의 외부IP]:5601 → 192.168.50.85:5601 형태

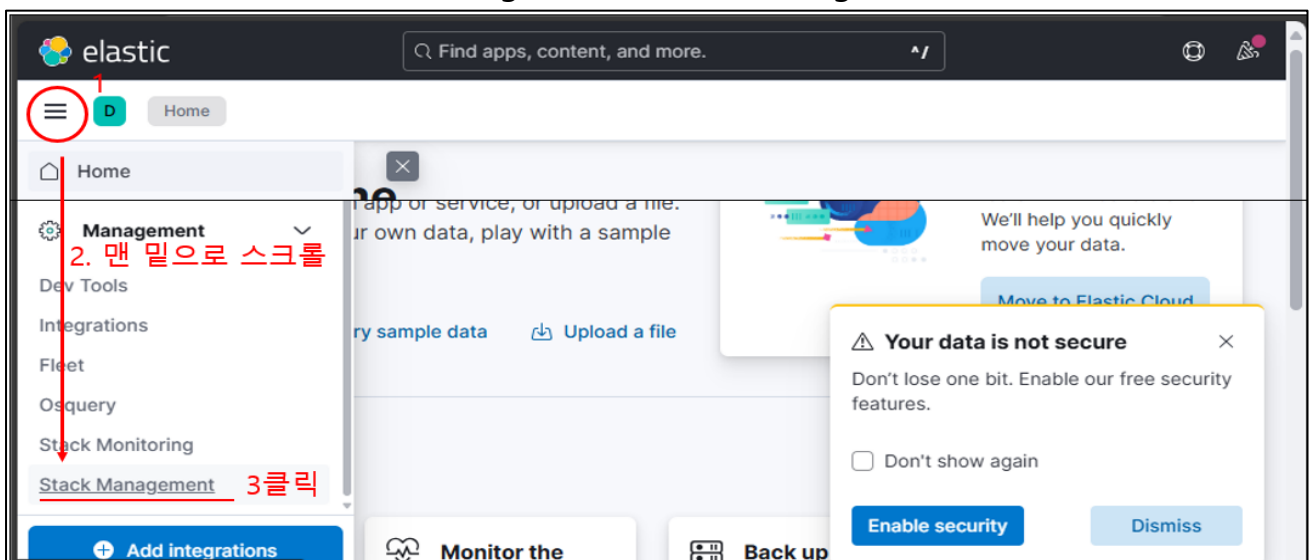


③. Explore my own 클릭하여 기본 페이지 진입

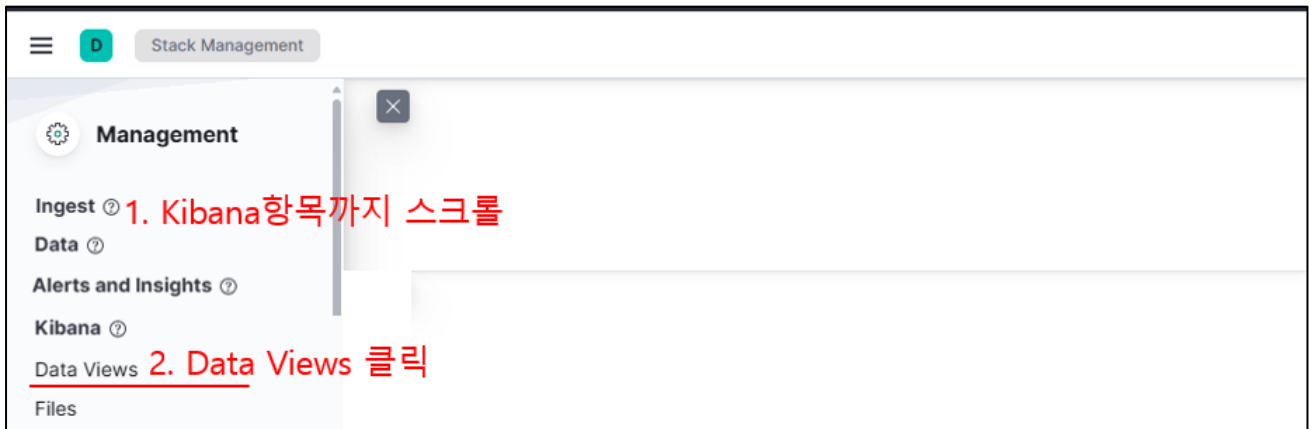


흐름19. Rsyslog 로 수집된 정보 확인

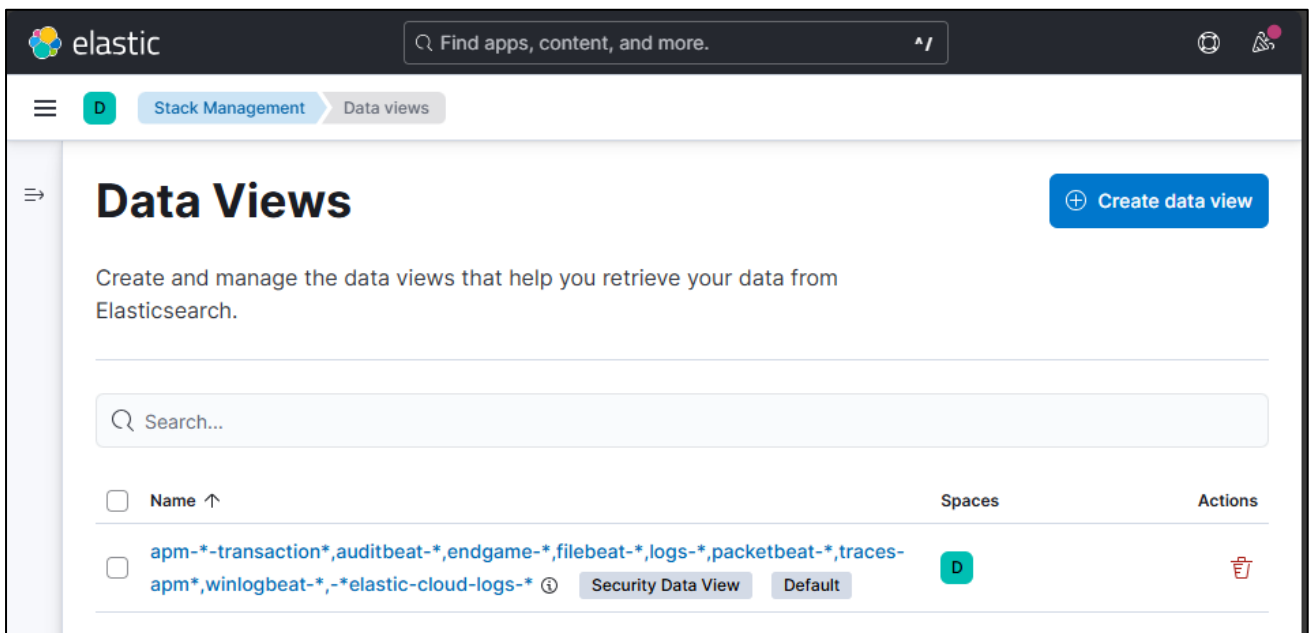
- ①. 외부에서 받은 로그를 보기 위해선 인덱스 파일을 생성해야 함
- ②. 인덱스 패턴 생성하기
 - 좌측 메뉴에서 Management → Stack Management 이동



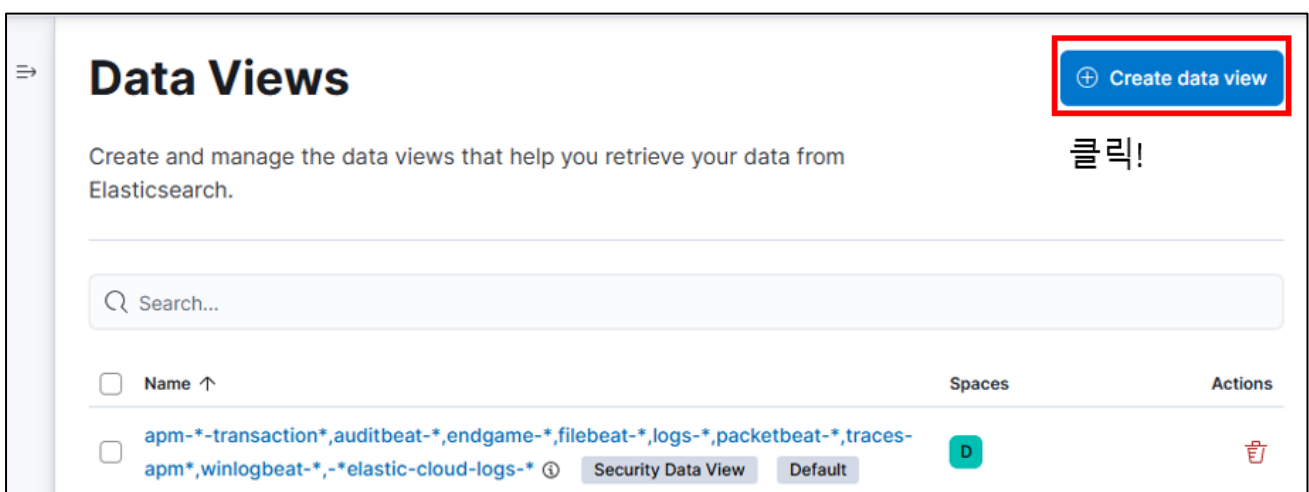
- Kibana → Data Views 클릭 → Create data view



- 진입 화면



- Create data view 클릭하여 인덱스 생성페이지 오픈



- Name 항목은 임의 지정. Index pattern은 remote-syslog-*

Create data view

Name: RsyslogS 임의 지정 가능

Index pattern: remote-syslog-* 임의 지정 불가능

Timestamp field: @timestamp

Select a timestamp field for use with the global time filter.

[Show advanced settings](#)

[Close](#) [Save data view to Kibana](#)

- 생성 확인됨

RsyslogS

Index pattern: remote-syslog-* Time field: @timestamp

Fields (32) Scripted fields (0) Field filters (0) Relationships (0)

Search

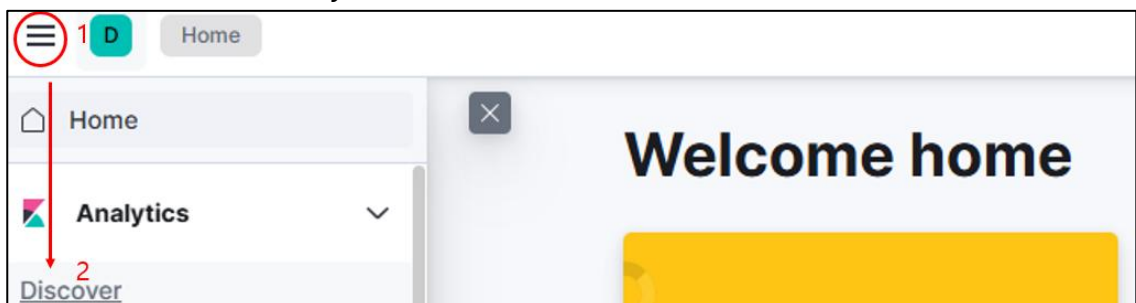
Name	Type	Format	Searchable	Aggregatable	Excluded
@timestamp	date		•	•	
@version	text		•		
@version.keyword	keyword		•	•	
_id	_id		•		
_ignored	_ignored		•	•	
_index	_index		•	•	
_score					
_source	_source				
event.original	text		•		
event.original.keyword	keyword		•	•	

Rows per page: 10

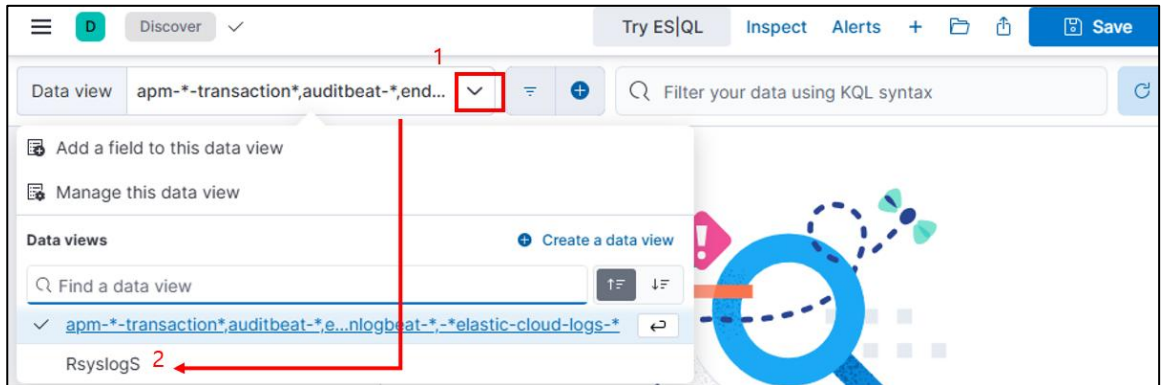
✓ Saved
"RsyslogS"

③. 로그가 정상적으로 수집되는지 확인

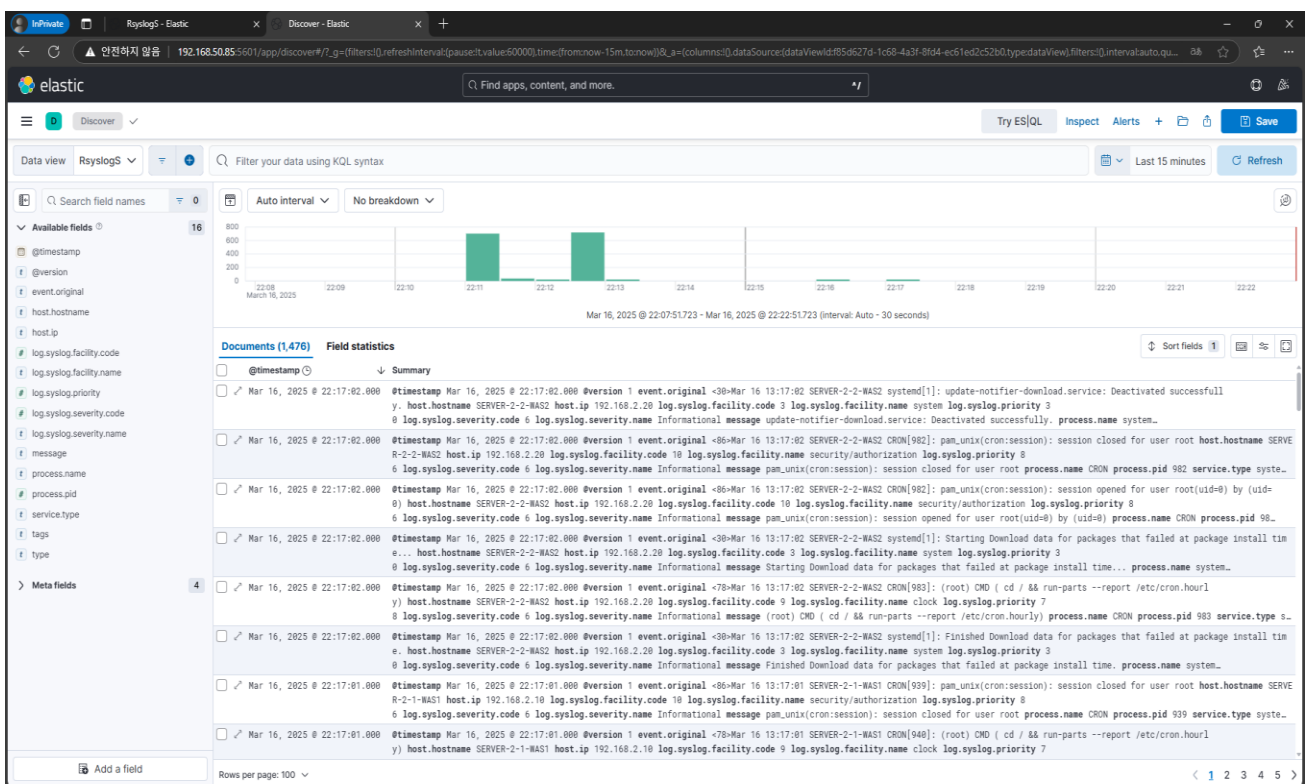
- 좌측 메뉴에서 Analytics → Discover 선택



- Data view에서 방금 전 생성한 RsyslogS 클릭
fhrlr



- 정상 기록 확인 가능



흐름20. 로깅 정책 적용하기

①. 로깅 정책 확인

- 중앙 집중식 로그 관리: 모든 서버의 로그는 **BastionHost**에서 수집/관리
 - 로그 운영 원칙
 1. 필요한 로그만 선별/저장하여 디스크 낭비 최소화
 2. 보존 기간을 정의하여 불필요한 로그 자동 삭제
 3. 보안 및 운영 효율을 고려한 정책 수립

- 로그 수집 범위

로그 유형	설명	수집 대상 서버
시스템 리소스 로그	CPU 사용률, 메모리 사용량, Disk I/O, 네트워크 사용량	모든 서버
네트워크 로그	방화벽(ipTables, firewalld) 로그, SSH 로그인 시도 (sshd 로그)	
WEB 서버 log	Access 로그, Error 로그, SSL 로그	Server 1 : WEB 서버, Nginx
WAS 서버 log	Application 로그, Exception 로그, Dependency 오류 로그	Server 2-1 & 2-2 : WAS 서버, Apache+php
DB 서버 log	Connection 로그, Transaction 로그, 오류 로그	Server 3 : DB 서버, MariaDB

- 로그 보존 정책

로그 유형	보존 기간
웹 서버 로그	3개월 보관 후 자동 삭제
애플리케이션 서버 로그	6개월 보관
DB 서버 로그	1년 보관
시스템 리소스 로그	6개월 보관
네트워크 로그	1년 보관

②. 로그 정책에 따른 설정 추가/수정 및 적용 (Logstash 설정)

- 웹, WAS, DB, 방화벽, SSH 로그만 저장하고 나머지는 삭제
- 설정 파일 위치: /etc/logstash/conf.d/remote-syslog.conf

<pre> sudo vim etc/logstash/conf.d/remote-syslog.conf input{ syslog{ port => 5140 type } } filter { if [program] =~ /nginx/ { mutate { add_tag => ["web_log"] } } else if [program] =~ /php/ { mutate { add_tag => ["was_log"] } } else if [program] =~ /mariadb/ { mutate { add_tag => ["db_log"] } } } </pre>

```

else if [program] =~ /(iptables|firewalld)/ {
    mutate { add_tag => ["firewall_log"] }
}
else if [program] =~ /sshd/ {
    mutate { add_tag => ["ssh_log"] }
}
else {
    drop {} # 필요 없는 로그는 제거
}
}

output {
  if "web_log" in [tags] {
    elasticsearch { hosts => ["http://localhost:9200"] index => "web-logs-%{+YYYY.MM.dd}" }
  }
  else if "was_log" in [tags] {
    elasticsearch { hosts => ["http://localhost:9200"] index => "was-logs-%{+YYYY.MM.dd}" }
  }
  else if "db_log" in [tags] {
    elasticsearch { hosts => ["http://localhost:9200"] index => "db-logs-%{+YYYY.MM.dd}" }
  }
  else if "firewall_log" in [tags] {
    elasticsearch { hosts => ["http://localhost:9200"] index => "firewall-logs-%{+YYYY.MM.dd}" }
  }
  else if "ssh_log" in [tags] {
    elasticsearch { hosts => ["http://localhost:9200"] index => "ssh-logs-%{+YYYY.MM.dd}" }
  }
  stdout { codec => rubydebug }
}

```

● 설정 반영

```

#문법 확인
sudo /usr/share/logstash/bin/logstash --config.test_and_exit
sudo systemctl restart logstash
sudo systemctl status logstash

```

③. Elasticsearch 인덱스 정리

로그 유형	Elasticsearch 인덱스	수집 대상 서버	보존 기간
시스템 리소스 로그	system-resource-logs-*	모든 서버	6개월
네트워크 로그	firewall-logs-*, ssh-logs-*	모든 서버	1년
웹 서버 로그	web-logs-*	WEB 서버 (Nginx)	3개월
WAS 서버 로그	was-logs-*	WAS 서버 (Apache+PHP)	6개월
DB 서버 로그	db-logs-*	DB 서버 (MariaDB)	1년

흐름21. 서버 헬스체크 플러그인(Metricbeat) 설치

①. Metricbeat 설치 확인

- 버전확인 명령어:

②. Metricbeat 설정 파일 수정(WEB / WAS / DB 서버 및 bastion)

- 설정파일 위치: /etc/metricbeat/metricbeat.yml

sudo vim /etc/metricbeat/metricbeat.yml
output.elasticsearch: hosts: ["192.168.4.10:9200"] # ELK 서버의 Elasticsearch 주소
setup.kibana: host: "192.168.4.10:5601" # Kibana 서버 주소
##추가적으로 BastionHost에서 적용 setup.dashboards.enabled: false
===== Outputs ===== # Configure what output to use when sending the data collected by the beat. # ----- Elasticsearch Output ----- output.elasticsearch: # Array of hosts to connect to. hosts: ["192.168.4.10:9200"] # Performance preset - one of "balanced", "throughput", "scale", # "latency", or "custom". preset: balanced
===== Kibana ===== # Starting with Beats version 6.0.0, the dashboards are loaded via the Kibana API. # This requires a Kibana endpoint configuration. setup.kibana: host: "192.168.4.10:5601" # Kibana 서버 주소
===== Dashboards ===== # These settings control loading the sample dashboards to the Kibana index. Loading # the dashboards is disabled by default and can be enabled either by setting the # options here or by using the `setup` command. setup.dashboards.enabled: false # The URL from where to download the dashboard archive. By default, this URL

③. Metricbeat 시스템 모듈 활성화

- 설정적용

sudo metricbeat modules enable system
sudo metricbeat setup

④. 각 서버에서 metricbeat 값 정상적으로 전송 확인

sudo metricbeat test output

```

elasticsearch: http://192.168.4.10:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: 192.168.4.10
    dial up... OK
  TLS... WARN secure connection disabled
  talk to server... OK
  version: 8.17.3

```

⑤. BastionHost에서 Elasticsearch에 데이터가 들어오는지 확인

```

curl -XGET "http://192.168.4.10:9200/_cat/indices?v" | grep metricbeat

```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
100	2800	0 2800	0 0	162k	0	--:--:--	170k
green	open	.ds-metricbeat-8.17.3-2025.03.17-000001					
101.2mb		101.2mb					

⑥. 웹페이지에서 확인

- 192.168.4.10:5601 접속
- <http://192.168.4.10:5601/app/management/kibana/dataViews> 진입
- Create Data View 클릭 후 metricbeat-* 추가

Create data view

Name

Index pattern

Timestamp field

Select a timestamp field for use with the global time filter.

✓ Your index pattern matches 1 source.

All sources Matching sources

metricbeat-8.17.3 Data stream

Rows per page: 10

MetricBeat

Index pattern: `metricbeat-*` Time field: `@timestamp`

Fields (6498) Scripted fields (0) Field filters (0) Relationships (0)

Search

Field type: 17 Schema type: 2 Refresh Add field

Name	Type	Format	Searchable	Aggregatable	Excluded
@timestamp	date		•	•	
_id	_id		•		
_ignored	_ignored		•	•	
_index	_index		•	•	
_score					
_source	_source				
activemq.broker.connections.count	long		•	•	
activemq.broker.consumers.count	long		•	•	
activemq.broker.mbean	keyword		•	•	
activemq.broker.memory.broker.pct	scaled_float		•	•	

Rows per page: 10

⑦. Dashboard에서 시각화된 데이터 보기

- Dashboard 메뉴 접근
 - Metricbeat System 검색

Dashboards

Metricbeat System

Recently viewed

Tags

Created by

Create dashboard

☐	Name, description, tags	Last updated	Actions
☐	[Metricbeat System] Host overview ECS Overview of host metrics	3 hours ago	i
☐	[Metricbeat System] Overview ECS Overview of system metrics	3 hours ago	i
☐	[Metricbeat System] Host Services Overview Overview of services on an individual host.	3 hours ago	i
☐	[Metricbeat System] Containers overview ECS Overview of container metrics	3 hours ago	i

Rows per page: 20

<

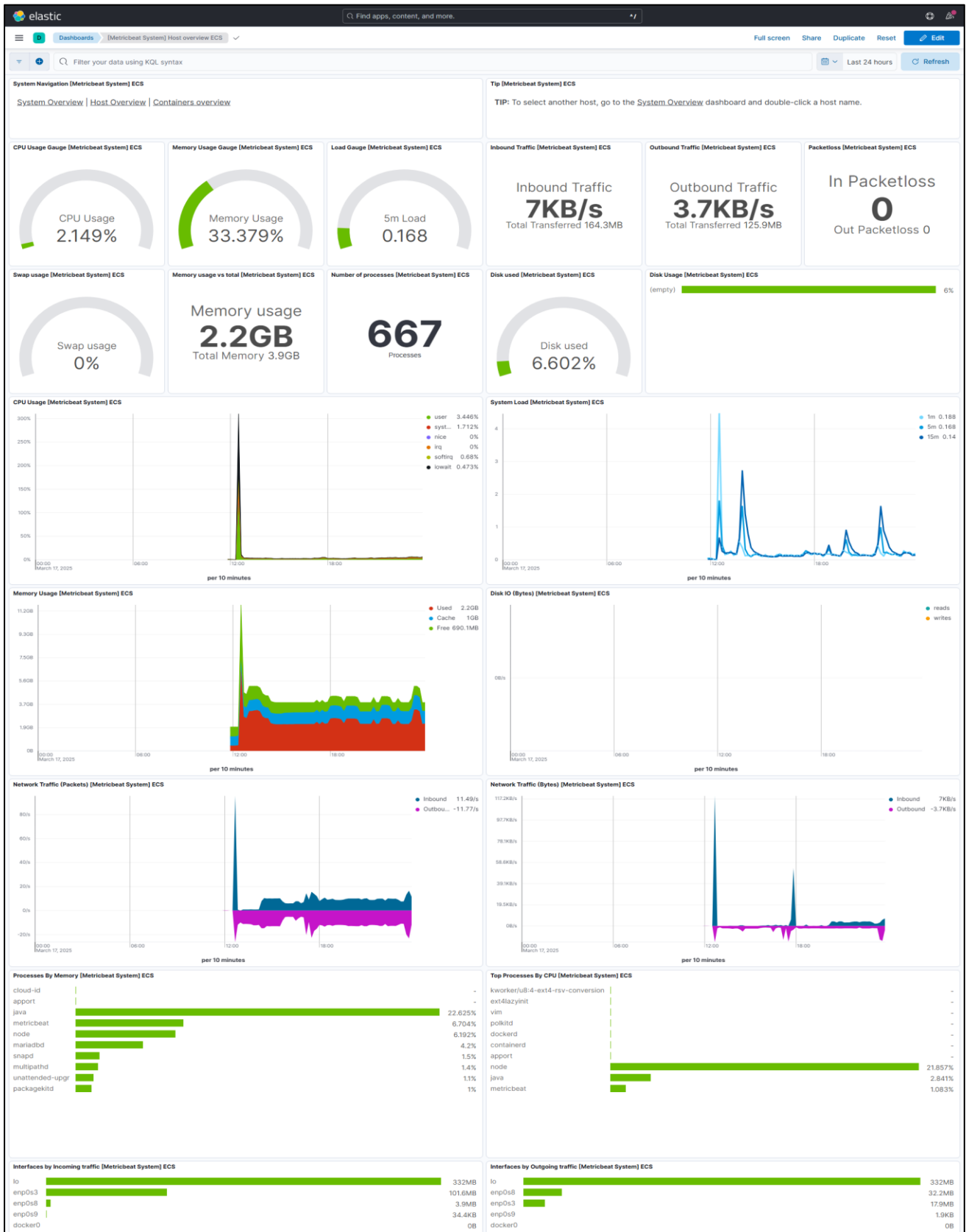
1

>

- **[Metricbeat System] Host Services Overview** 대시보드 열기
 - 호스트 전체 상태를 한눈에 확인
- [Metricbeat System] Overview ECS
 - 시스템 메트릭 개요. vsphere 모듈이 활성화 되어야 함 → 현재는 못봄
- [Metricbeat System] Host Services Overview
 - 개별 호스트에서 실행 중인 서비스 상태 확인
 - system 모듈의 서비스(metricset: service) 설정이 켜져있어야 함.
 - 즉, 현재는 못봄
- [Metricbeat System] Containers overview ECS
 - 컨테이너 메트릭 모니터링 → 현재는 못봄

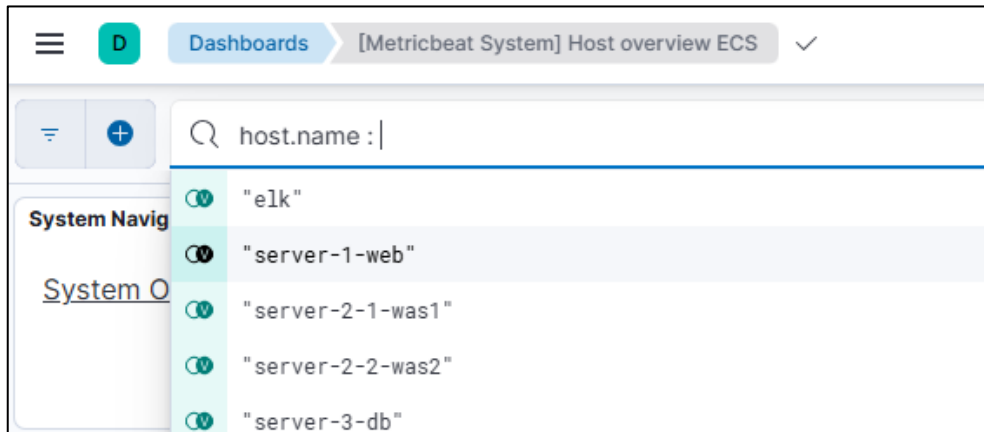
<다음장 계속>

⑧. 현재 확인 가능한 대시보드 열어서 리소스 상태 확인

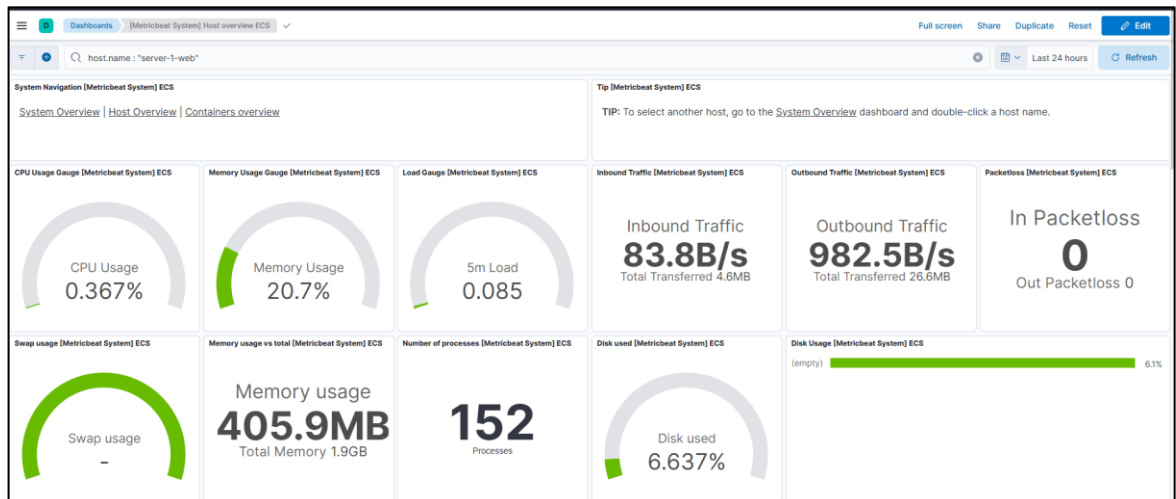


* 위 상태는 모든 정보가 혼합된 상태. 좀 더 분리해서 봐야함

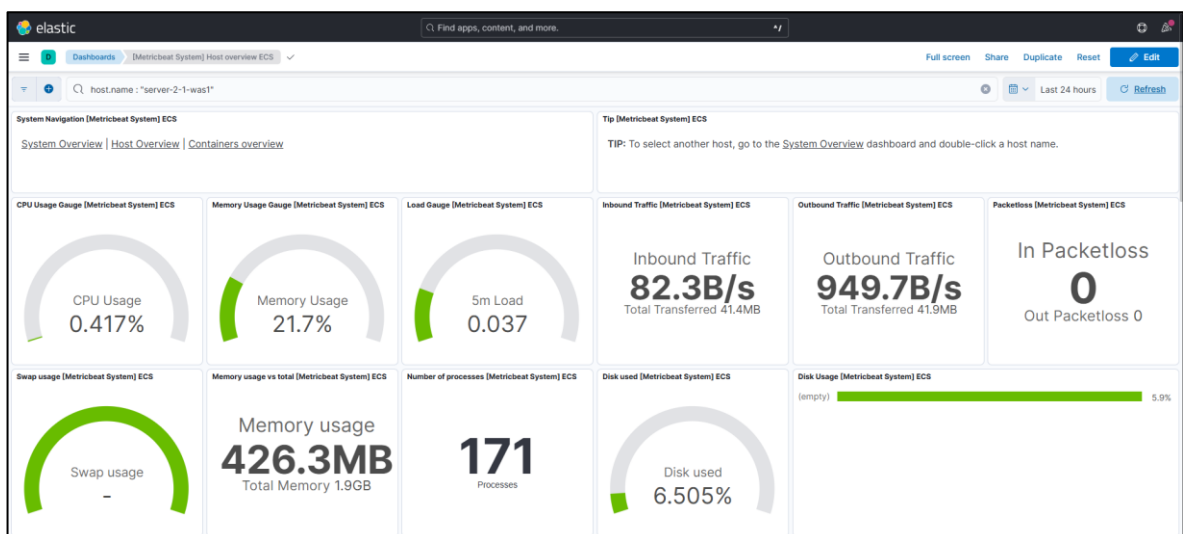
- 각 호스트 이름을 넣어 상태 확인(필터창에 hsoast.name 까지 치면 알려준다)



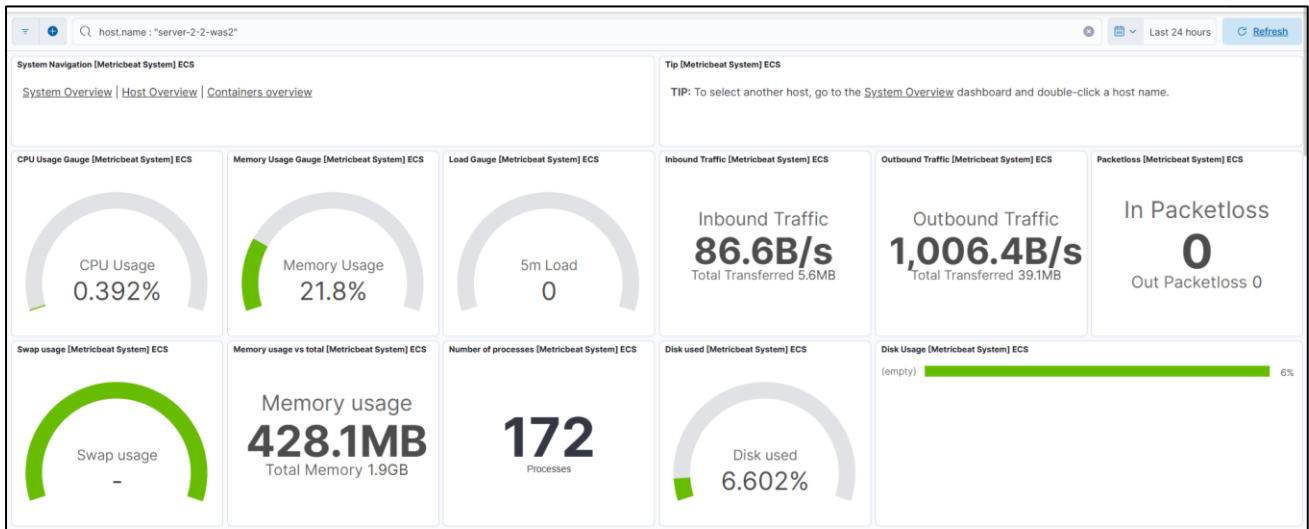
- WEB: "server-1-web"



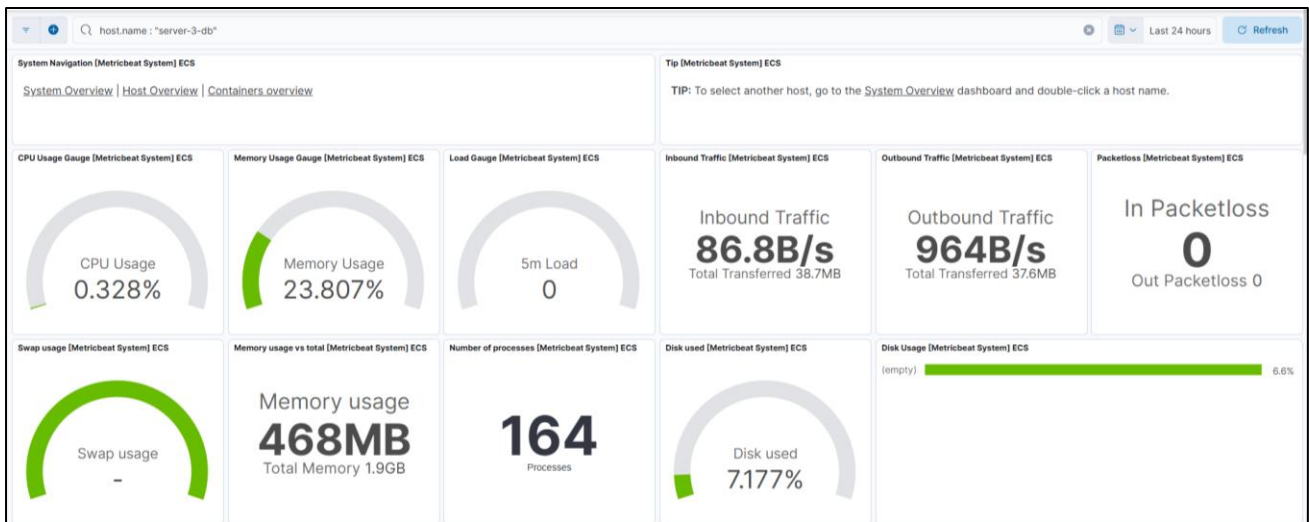
- WAS1: "server-2-1-was1"



- WAS2: "server-2-2-was2"



- DB: "server-3-db"



- bastionhost: "elk"

